

I.E.S. Los Montecillos

MINERVA

DESARROLLO DE APLICACIONES MULTIPLATAFORMA

Curso 2025/26



FRANCISCO PACHECO GÓMEZ
DANIEL FLORES JIMENEZ



ANEXO I: CALIFICACIÓN

Reunidos, en la fecha indicada, el Tribunal Evaluador, compuesto por los Profesores abajo firmantes, y vista la defensa del Proyecto Integrado denominado:

Presentado por D/D^a:

Se otorga la calificación de: _____

Y para que así conste, se firma en Coín a _____ de _____ de 20__

El Tribunal Evaluador

D/D^a _____ D/D^a _____ D/D^a _____



Agradecimientos

Este proyecto está dedicado a Rafael Ruiz, Javier Ordóñez y Luis Turmo, quienes han sido mis mentores durante estos dos años de formación en Desarrollo de Aplicaciones Multiplataforma.

Gracias por la paciencia en el aula, por resolver mis dudas interminables y por impulsarme a dar siempre un paso más allá en cada línea de código. Haber llegado a diseñar y desarrollar un sistema como este no habría sido posible sin vuestra calidad humana y profesional. Gracias por ayudarme a construir los cimientos de mi carrera.



Índice

CAPÍTULO 1: INTRODUCCIÓN Y JUSTIFICACIÓN.....	7
1.1. Contexto y Problema.....	7
1.2. Necesidad Identificada.....	7
1.3. Justificación de la Solución Propuesta.....	7
CAPÍTULO 2: OBJETIVOS.....	9
2.1. Objetivo General.....	9
2.2. Objetivos Específicos.....	9
2.3. Alcance (Inclusiones).....	9
2.4. Limitaciones y Exclusiones.....	10
CAPÍTULO 3: ANTECEDENTES.....	12
3.1 Introducción.....	12
3.2 Estudios de referencia.....	12
3.2.1 Estudio de Bem-Haja et al. (2025): Predicción Temprana en Educación Primaria	12
3.2.2 Estudio de George et al. (2025): Explicabilidad en Educación Superior.....	13
3.3 Conclusiones.....	14
Fundamentación Técnica.....	14
Directrices de Implementación.....	14
3.4 Valor Añadido Propuesto.....	15
CAPÍTULO 4: ARQUITECTURA DEL PROYECTO.....	16
4.1. Tecnologías y Frameworks.....	16
4.1.1 Frontend.....	16
4.1.2 Backend y procesamiento de datos.....	17
4.1.3 Base de datos.....	18
4.1.4 Machine Learning.....	19
4.1.5 Datasets.....	20
4.1.6 Testing.....	20
4.2. Análisis de la Solución.....	21
4.2.1. Actores del Sistema.....	21
4.2.2. Requisitos Funcionales.....	22
4.2.3. Requisitos No Funcionales.....	23
4.3. Modelado de Casos de Uso.....	24
4.3.1. Diagrama de casos de uso.....	24
4.3.2. Especificación de Casos de uso Clave.....	24
CU-01: Insertar datos de alumno.....	24
CU-02: Predecir probabilidad de aprobado.....	25



CU-03: Consultar Listado de Predicciones.....	26
CU-04: Importar datos.....	27
4.4. Arquitectura General del Sistema.....	28
4.5. Aplicación Web.....	29
4.5.1. Estructura de páginas.....	29
4.5.2. Dashboard de predicción.....	29
4.6. Aplicación Android.....	30
4.6.1. Tecnologías y versiones.....	30
4.6.2. Arquitectura (Clean Architecture).....	31
4.6.3. Navegación y pantallas.....	32
4.6.4. Capa de red y resiliencia.....	34
4.6.5. Autenticación y seguridad.....	35
4.6.6. Perfil de usuario editable.....	36
4.6.7. Empaquetado y distribución.....	36
4.7. API REST.....	36
4.8. Modelo ML.....	37
4.8.1. Pipeline de entrenamiento.....	38
4.8.2. Catálogo de Resultados de Aprendizaje.....	40
4.8.3. Conjunto de características (features).....	40
4.8.4. Niveles de riesgo.....	41
4.8.5. Aprendizaje incremental.....	43
4.8.6. Advertencia metodológica.....	43
4.9. Procesamiento de Lenguaje Natural del Feedback.....	44
4.10. Despliegue, Infraestructura y Control de Versiones.....	45
CAPÍTULO 5 : FASES DE TRABAJO.....	46
5.1 Modelo de Desarrollo.....	46
5.2. Viabilidad y Planificación Inicial.....	46
5.2.1. Viabilidad Técnica.....	46
5.2.2. Viabilidad Económica.....	47
5.2.3. Viabilidad Temporal.....	47
5.2.4. Viabilidad Legal.....	49
CAPÍTULO 6: PRUEBAS DE FUNCIONAMIENTO.....	51
6.1. Pruebas de Integración General.....	51
6.2. Prueba de Inserción de Datos de Alumno (CU-01).....	52
6.3. Prueba de Predicción Automática (CU-02).....	53
6.4. Prueba de Consulta de Predicciones (CU-03).....	53
6.5. Prueba de Importación Masiva (CU-04).....	54
6.6. Pruebas de Rendimiento.....	55
6.7. Pruebas de Seguridad.....	56
6.8. Pruebas del Modelo de Machine Learning.....	57
Métricas generales.....	57
Validación del Modelo.....	60
Hiperparámetros Óptimos.....	61
Importancia de las Características.....	61



6.9. Pruebas de Usabilidad.....	63
6.10. Pruebas de Fiabilidad y Recuperación.....	64
6.11. Conclusiones del Capítulo.....	65
CAPÍTULO 7: PROBLEMAS ENCONTRADOS.....	66
P-01: Análisis de Feedback con NLP Básico.....	66
P-02: Tiempos de Arranque Lento en Render (Cold Start).....	67
P-03: Precisión del Modelo en Datos Reales (Pendiente de Validación).....	68
P-04: Ausencia de Tests Automatizados en el Backend.....	68
P-05: Documentación de Código Limitada.....	69
P-06: Polarización en los Porcentajes de Probabilidad Predichos.....	70
CAPÍTULO 8: MEJORAS PROPUESTAS.....	72
PRIORIDAD ALTA (Implementar en v0.2.0).....	73
M-01: Exportación de Informes (PDF / CSV).....	73
M-05: Explicabilidad de Predicciones (SHAP / LIME).....	74
M-08: Suite de Tests Automatizados (pytest).....	75
M-03: Integración con Plataformas Oficiales (Séneca / Moodle).....	76
PRIORIDAD MEDIA (Implementar en v0.3.0).....	77
M-02: NLP Avanzado para Feedback del Profesor.....	77
M-06: Gráficos de Evolución Temporal por Alumno.....	78
M-10: Dashboard de Administrador.....	79
PRIORIDAD BAJA (Implementar en v1.0.0+).....	80
M-07: Google Play + Modo Offline.....	80
M-04: Validación con Datos Reales del Centro.....	81
M-09: Documentación de Código (Docstrings).....	82
8.1. Panel Analítico Avanzado.....	83
CAPÍTULO 9: CONCLUSIONES.....	85
9.1. Resumen del proyecto.....	85
9.2. Soluciones aportadas.....	87
9.3. Valoración personal del desarrollo.....	89
9.4. Uso profesional y futuro del sistema.....	92
9.5. Conclusión General.....	94
CAPÍTULO 10: BIBLIOGRAFÍA.....	96
ANEXOS.....	97
ANEXO I : Diagrama de casos de uso.....	97
Entrenamiento Inicial.....	97
Flujo de actualización diaria.....	97
Flujo de entrenamiento incremental.....	99
ANEXO II: Diagrama de Gantt.....	100
ANEXO III: Mocks de los clientes.....	100



CAPÍTULO 1: INTRODUCCIÓN Y JUSTIFICACIÓN

1.1. Contexto y Problema

En el ámbito educativo actual, los centros de enseñanza buscan nuevas estrategias para mejorar el rendimiento académico y reducir el fracaso escolar. Sin embargo, la detección temprana de alumnos en riesgo de suspender sigue siendo un reto.

Los métodos tradicionales de seguimiento — como las calificaciones parciales o la observación directa del profesorado — suelen detectar los problemas demasiado tarde, cuando las medidas de apoyo ya tienen un impacto limitado.

Por ello, surge la necesidad de contar con herramientas tecnológicas que permitan **anticipar el rendimiento de los alumnos y apoyar la toma de decisiones pedagógicas** de forma proactiva.

1.2. Necesidad Identificada

Actualmente, muchos centros carecen de sistemas que integren de manera automática los datos académicos, la asistencia, la participación y otros indicadores de rendimiento.

Esta falta de análisis predictivo impide detectar con precisión qué alumnos pueden tener dificultades, y limita la capacidad del profesorado para actuar preventivamente.

El proyecto busca cubrir esta necesidad mediante una aplicación denominada **Minerva** (Modelo **IN**teligente de Evaluación de Rendimiento y Valoración Académica), que utiliza técnicas de *machine learning* para analizar los datos de cada alumno y predecir la probabilidad de que apruebe una asignatura concreta.

1.3. Justificación de la Solución Propuesta

La solución propuesta se justifica por su potencial para optimizar el proceso educativo mediante el uso inteligente de datos, superando las limitaciones de privacidad y



disponibilidad de datos reales mediante la creación de un dataset sintético representativo. Minerva aprovechará:

- Datasets públicos educativos de rendimiento académico
- Parámetros y Resultados de Aprendizaje (RAs) establecidos por la Junta de Andalucía
- Técnicas de generación de datos sintéticos para ampliar y diversificar el dataset

Este enfoque permitirá entrenar un modelo robusto sin comprometer datos sensibles de alumnos reales, manteniendo la representatividad del contexto educativo andaluz.

CAPÍTULO 2: OBJETIVOS

2.1. Objetivo General

Desarrollar una aplicación que, mediante técnicas de machine learning, permita predecir la probabilidad de que un alumno apruebe una asignatura concreta, con el fin de facilitar la detección temprana de alumnos en riesgo académico y mejorar la toma de decisiones pedagógicas.

2.2. Objetivos Específicos

1. Diseñar e implementar un modelo de predicción basado en machine learning que analice variables académicas y de comportamiento del alumno.
2. Diseñar un generador de datos sintéticos basado en datos anonimizados públicos, RAs de la Junta de Andalucía y parámetros educativos realistas
3. Permitir la actualización dinámica de las predicciones conforme el alumno genera nuevos datos (RA, prácticas, exámenes, asistencia).
4. Desarrollar una interfaz que muestre los resultados de manera comprensible para el profesorado y la dirección.
5. Proporcionar métricas que ayuden a identificar patrones que influyen en el éxito o fracaso académico.
6. Ofrecer acceso multiplataforma al sistema mediante una aplicación web y una aplicación móvil Android nativa, ambas consumiendo una misma API REST.

2.3. Alcance (Inclusiones)



- Implementación de un modelo de predicción del rendimiento académico basado en técnicas de machine learning, centrado inicialmente en una asignatura concreta del curso o ciclo formativo.
- Carga y procesamiento de datos académicos (notas, asistencia, prácticas, RA, feedback del profesor).
- Generación de predicciones actualizadas para cada alumno y asignatura.
- Interfaz orientada a profesorado y dirección del centro, con visualización de resultados y alertas de riesgo.
- Posibilidad de actualizar el modelo con nuevos datos históricos.
- Plataformas objetivo: El sistema está orientado actualmente a dos plataformas cliente: una aplicación web accesible desde navegadores modernos y una aplicación móvil Android. Ambas consumen los mismos servicios expuestos por la API REST y utilizan mecanismos unificados de autenticación y persistencia de datos. Como trabajo futuro, se prevé la incorporación de una aplicación para iOS, manteniendo la interoperabilidad con la infraestructura existente y la sincronización centralizada de la información.

2.4. Limitaciones y Exclusiones

- No se incluirá una plataforma de gestión completa de alumnos (el sistema se centrará únicamente en la predicción).
- El modelo se entrenará con datos sintéticos en lugar de datos reales del centro.
- La precisión del modelo dependerá de la calidad y volumen de los datos disponibles.
- No se contempla la integración directa con sistemas externos (por ejemplo, plataformas educativas oficiales) en esta primera versión.
- Las predicciones se ofrecerán como apoyo al profesorado, sin sustituir su criterio pedagógico.



- No se incluye el registro ni la gestión de usuarios (CRUD) desde la propia plataforma: el alta de profesores se realiza de forma administrativa en Supabase Auth, y la aplicación únicamente permite iniciar sesión a usuarios previamente dados de alta.
- El feedback del profesor se incorpora como una columna del archivo CSV del grupo; en esta versión no es posible redactarlo ni editarlo desde la interfaz web o la aplicación Android.

CAPÍTULO 3: ANTECEDENTES

3.1 Introducción

El auge de la inteligencia artificial ha permeado todos los sectores, siendo el educativo uno de los más transformados. Entre las múltiples aplicaciones, los sistemas predictivos de rendimiento académico han demostrado un potencial significativo. Para contextualizar este proyecto, se analizan dos estudios recientes que representan aproximaciones complementarias en este campo.

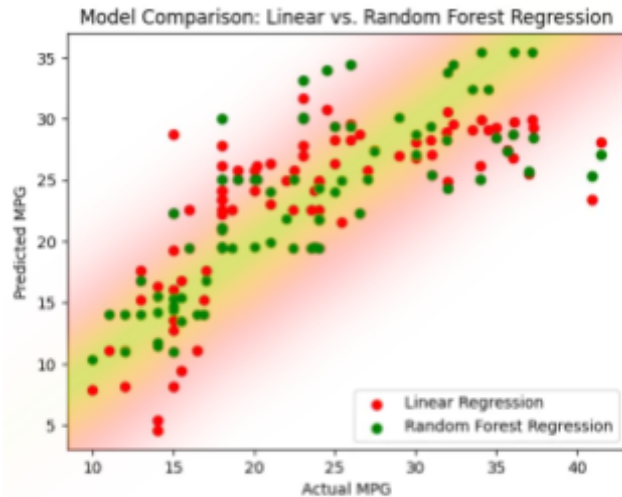
3.2 Estudios de referencia

3.2.1 Estudio de Bem-Haja et al. (2025): Predicción Temprana en Educación Primaria

La investigación portuguesa liderada por Bem-Haja examinó la viabilidad de predecir el rendimiento académico en 2,549 estudiantes de primer grado. Utilizando técnicas de machine learning como Random Forest, Gradient Boosting y Extra Trees, lograron una precisión excepcional del 99.7% en la identificación de estudiantes en riesgo de suspender.

Aportaciones clave:

- Demostró la efectividad de algoritmos basados en árboles para predicción educativa
- Identificó que las variables más predictivas incluyen la percepción docente sobre atención y organización, habilidades de lectoescritura temprana, y el nivel educativo parental
- Validó que es posible intervenir preventivamente mediante screening inicial



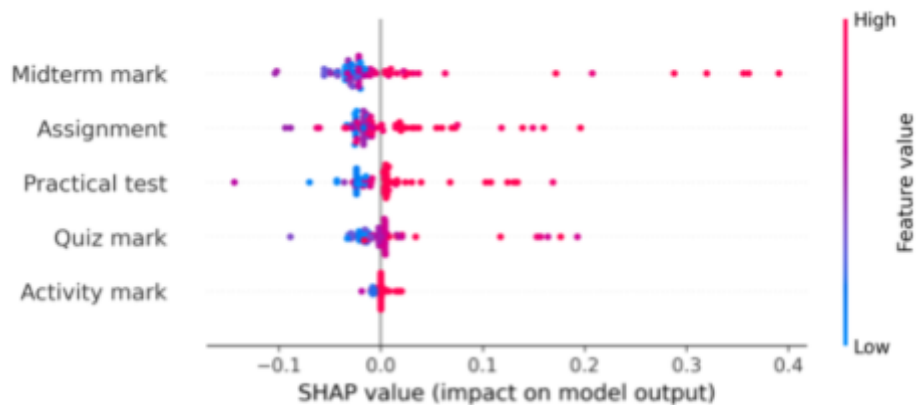
"Figura 3.2: Comparativa entre los modelos de regresión lineal y random forest en la predicción del consumo de combustible (medido en millas por galón) para distintos vehículos. Cada punto representa un vehículo; cuanto más cerca esté de la zona verde, menor es el error de predicción y, por tanto, mayor la precisión del modelo en ese caso."

3.2.2 Estudio de George et al. (2025): Explicabilidad en Educación Superior

Por su parte, la investigación de George y colaboradores en Omán se centró en la aplicación de técnicas de Explainable AI (XAI) para predecir el rendimiento universitario. Con un 92% de precisión usando Random Forest, incorporaron SHAP y LIME para hacer transparentes las predicciones.

Aportaciones clave:

- Enfatizó la importancia de la explicabilidad en sistemas educativos
- Identificó que exámenes parciales y assignments son los predictores más influyentes
- Abordó críticamente los riesgos de sesgo algorítmico y estrategias de mitigación



"Figura 3.1: Análisis de importancia de características mediante valores SHAP. El eje horizontal representa el impacto en la predicción (valores SHAP positivos aumentan la probabilidad de aprobado, negativos la disminuyen). El color indica el valor de cada característica, donde azul (low) representa valores bajos y rojo (high) valores altos. Cada punto corresponde una observación del conjunto de datos."

3.3 Conclusiones

Fundamentación Técnica

Ambos estudios validan el enfoque de este proyecto, demostrando que:

- Los algoritmos de Random Forest y Extra Trees son óptimos para predicción académica
- Es posible lograr alta precisión manteniendo la interpretabilidad
- El balance entre efectividad y transparencia es técnicamente viable

Directrices de Implementación

Para el desarrollo de este sistema de predicción de aprobados, se derivan las siguientes directrices:

- Selección algorítmica: Implementar Random Forest Classifier como algoritmo principal, dada su efectividad demostrada, con SGDClassifier como alternativa para escenarios que prioricen explicabilidad.
- Arquitectura explicable: Integrar SHAP y LIME desde la fase de diseño, garantizando que cada predicción sea interpretable por educadores y estudiantes.



- Variables predictoras: Incluir tanto indicadores académicos formales (evaluaciones parciales) como métricas de proceso (participación, consistencia).
- Enfoque ético: Establecer protocolos para detección y mitigación de sesgos, evitando la perpetuación de desigualdades existentes.

3.4 Valor Añadido Propuesto

Este proyecto sintetiza las lecciones de ambos estudios, proponiendo un sistema que combine la precisión predictiva del estudio de Bem-Haja junto con la transparencia algorítmica del trabajo de George.

La evidencia científica revisada no sólo valida la viabilidad del proyecto, sino que proporciona un roadmap técnico respaldado empíricamente para su desarrollo e implementación.

Además, incorpora el innovador enfoque de generación de datos sintéticos educativos. Esto permite:

- Superar barreras de privacidad en el acceso a datos reales
- Garantizar representatividad mediante parámetros educativos oficiales
- Facilitar replicabilidad en otros centros educativos
- Mantener rigor científico mediante validación cruzada con datasets públicos existentes



CAPÍTULO 4: ARQUITECTURA DEL PROYECTO

4.1. Tecnologías y Frameworks

4.1.1 Frontend

El sistema ofrece dos clientes que consumen la misma **API REST**: una aplicación web y una aplicación móvil nativa para Android.

Aplicación web. Para la interfaz web se ha seleccionado una combinación de tecnologías estándar que garantizan compatibilidad y responsividad:

HTML5 para la estructura de los formularios y tablas de alumnos

CSS3 (organizado en 16 hojas de estilo modulares) para el diseño responsive y estilos visuales adaptados a dispositivos móviles

JavaScript para la interactividad dinámica, la carga de archivos CSV, la validación de formularios y la comunicación asíncrona con el backend

Jinja2 como motor de plantillas para el renderizado dinámico de las vistas HTML desde **FastAPI**

Aplicación móvil (Android). Como cliente nativo se ha desarrollado una aplicación Android que permite al profesorado autenticarse, cargar un CSV y consultar las predicciones desde el móvil:

Kotlin como lenguaje principal y **Jetpack Compose** con Material 3 para una interfaz declarativa y moderna

Arquitectura limpia (**Clean Architecture**) en tres capas (data, domain, presentation) con patrón **MVVM** (Compose + ViewModel)



Hilt para la inyección de dependencias, **Navigation Compose** para la navegación y **DataStore** para la persistencia de la sesión

Retrofit + OkHttp + kotlinx.serialization para la comunicación con la API, con renovación automática del token de acceso

Autenticación directa contra **Supabase** Auth y envío del token (Bearer JWT) al backend

4.1.2 Backend y procesamiento de datos

El núcleo del sistema está desarrollado en Python, aprovechando su ecosistema para ciencia de datos:

Python como lenguaje principal por su versatilidad y amplias librerías para ML

FastAPI + **Uvicorn** para exponer servicios web **RESTful** que conecten los clientes con la lógica de negocio

Starlette SessionMiddleware para la gestión de sesiones del cliente web

Pandas y **NumPy** para el procesamiento eficiente de datasets y la transformación de características (features)

NumPy (generador default_rng) para la generación de datos sintéticos tabulares con correlaciones realistas entre variables

SDK de Supabase (supabase-py) para la autenticación y la persistencia de datos

PyJWT y **cryptography** para la verificación local de los tokens JWT (ES256) emitidos por Supabase

joblib / pickle para la serialización y carga del modelo entrenado (.pkl)



4.1.3 Base de datos

Para el almacenamiento persistente de datos académicos y predicciones se utiliza **Supabase**, una plataforma gestionada construida sobre **PostgreSQL**:

PostgreSQL (gestionado por Supabase) como sistema gestor de bases de datos relacional, ideal para datos estructurados de alumnos

Acceso a los datos mediante la **API PostgREST de Supabase**, sin necesidad de un conector de bajo nivel

Tablas principales: predicciones (histórico de predicciones por alumno) y perfiles (perfil del usuario, relación 1:1 con auth.users)

Seguridad a nivel de fila (Row Level Security) para garantizar que cada usuario solo accede a sus propios datos

Supabase Auth para la gestión de usuarios y la autenticación basada en JWT

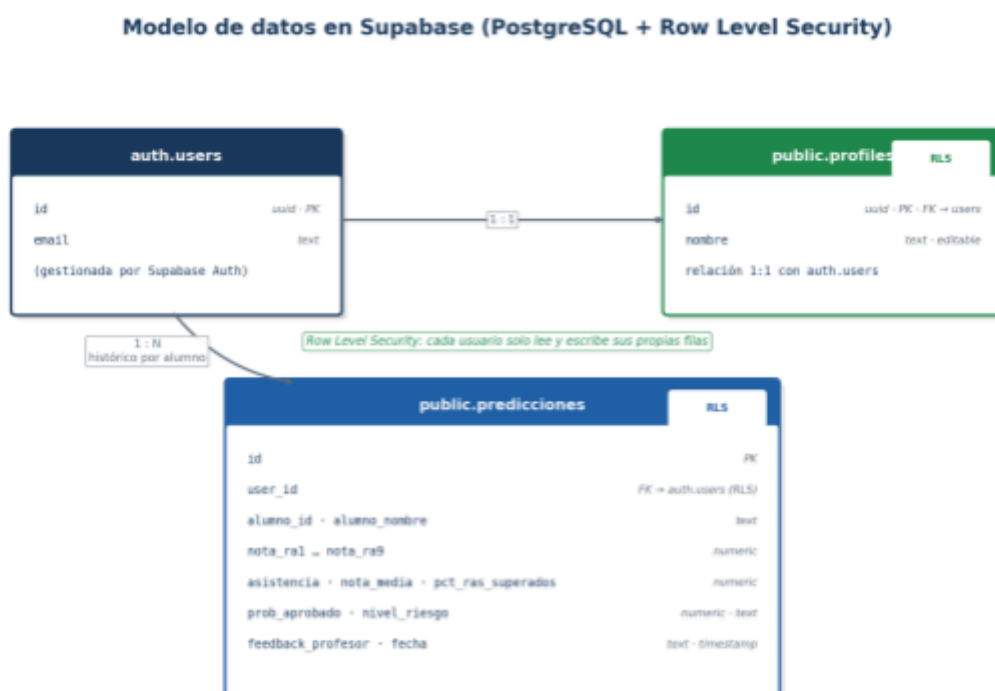


Figura 4.1.3. Modelo de datos en **Supabase**: perfiles en relación 1:1 con auth.users y predicciones como histórico 1:N por alumno, ambas protegidas con Row Level Security



4.1.4 Machine Learning

El componente predictivo utiliza librerías especializadas en machine learning:

scikit-learn como framework principal para algoritmos de clasificación supervisada

RandomForestClassifier como modelo principal de **predicción**, optimizado mediante **GridSearchCV** con validación cruzada estratificada (**StratifiedKFold**) sobre la métrica **F1**

SGDClassifier (con pérdida `log_loss`) como modelo secundario para el aprendizaje incremental mediante `partial_fit`, sin necesidad de reentrenamiento completo

Preprocesamiento de features con **SimpleImputer** (imputación de valores ausentes por mediana) y **StandardScaler** (normalización)

SMOTE (imbalanced-learn) previsto en el pipeline para el balanceo de clases cuando la clase minoritaria es inferior al 40 %. En la versión actual la librería no está incluida en las dependencias, por lo que el balanceo se delega en el parámetro `class_weight=balanced` del **RandomForest**

Conjunto de 23 features: las 9 notas de Resultados de Aprendizaje (RA), 3 features académicas base (asistencia, prácticas, feedback), 7 features derivadas (media, porcentaje de RAs superados, tendencia de progreso, gap teoría-práctica, etc.), un indicador de RA bloqueante y 4 features de contexto opcionales

Métricas de evaluación integradas para validar la precisión del modelo (accuracy, precision, recall, F1-score, kappa de Cohen y matriz de confusión), exportadas a un archivo `model_metrics.json`

4.1.5 Datasets

Datos totalmente sintéticos generados de forma programática con NumPy (1.000 alumnos por defecto), reproducibles mediante una semilla fija

Parámetros oficiales: RAs de la Junta de Andalucía por asignatura y nivel como referente curricular

Distribuciones estadísticas y correlaciones entre RAs definidas a partir de informes educativos y criterios de evaluación realistas (normativa de FP: media ≥ 5 , asistencia $\geq 75\%$, $\geq 60\%$ de RAs superados)



Figura 4.1.5. Matriz de correlación entre los 9 RAs del dataset sintético. Las correlaciones (0,68–0,81) reproducen el comportamiento esperado: un alumno con buenos RAs básicos tiende a superar los avanzados.

4.1.6 Testing

El sistema incorpora mecanismos específicos para garantizar la fiabilidad del código y los modelos predictivos:



Autoevaluación del modelo integrada en el script de entrenamiento, que calcula accuracy, **precision, recall, F1-score, kappa y matriz de confusión**, y valida automáticamente el cumplimiento de umbrales mínimos con un veredicto PASS/FAIL (modelo apto / no apto).

Validación cruzada (cross_val_score con StratifiedKFold, k=5) de scikit-learn para asegurar que el modelo generaliza correctamente a nuevos alumnos.

Pruebas unitarias con **JUnit** en la aplicación Android, en particular para el parser de CSV (CsvStudentParserTest), validando el correcto procesamiento de los datos de entrada.

Persistencia de las métricas en model_metrics.json para su consulta posterior a través del endpoint /modelo/metricas.

4.2. Análisis de la Solución

4.2.1. Actores del Sistema

Tabla de actores con su descripción y responsabilidades.

Actor	Descripción	Responsabilidades
Profesor	Usuario principal que utiliza el sistema para predecir el rendimiento de sus alumnos	- Introducir datos académicos de alumnos - Consultar predicciones de aprobado - Visualizar listados y evoluciones
Administrador	Usuario con permisos avanzados de gestión del sistema	- Entrenar/reentrenar modelos de ML - Mantener la base de datos - Configurar parámetros globales



4.2.2. Requisitos Funcionales

Lista numerada y priorizada de lo que el sistema **debe hacer**.

- **RF-01:** Gestión de Datos de Alumnos. El sistema debe permitir al profesor cargar los datos académicos de los alumnos (notas de RA, asistencia, prácticas y feedback) mediante archivo CSV. La edición y eliminación individual de alumnos queda fuera del alcance de esta versión, conforme a las exclusiones del apartado 2.4.
- **RF-02:** Entrenamiento del Modelo de Machine Learning. El sistema debe permitir al administrador entrenar o reentrenar el modelo de predicción con los datos disponibles en la base de datos.
- **RF-03:** Predicción de Aprobación. El sistema debe calcular automáticamente la probabilidad de aprobado para cada alumno basándose en sus datos y el modelo de machine learning.
- **RF-04:** Consulta de Predicciones. El sistema debe permitir al profesor visualizar un listado de alumnos con su probabilidad de aprobado y la evolución a lo largo del tiempo.
- **RF-05:** Procesamiento de Feedback. El sistema debe transformar el texto libre de feedback del profesor, proporcionado como columna del archivo CSV, en una puntuación numérica mediante reglas de NLP básicas.
- **RF-06:** Gestión de Usuarios. El sistema debe permitir la autenticación de profesores y administradores previamente dados de alta en Supabase Auth. El registro de nuevos usuarios se realiza de forma administrativa (consola de Supabase) y queda fuera del alcance de esta versión, al igual que la diferenciación de roles, prevista como mejora futura (M-10).
- **RF-07:** Exportación de Resultados. El sistema debe permitir exportar los resultados de las predicciones a un formato legible (como CSV o PDF).

4.2.3. Requisitos No Funcionales

- **RNF-01:** Usabilidad. La interfaz debe ser intuitiva, permitiendo a un profesor realizar las tareas principales (como introducir un alumno y ver predicciones) en menos de 3 pasos.
- **RNF-02:** Rendimiento. Las consultas principales a la base de datos deben tener un tiempo de respuesta inferior a 200 ms. Además, el modelo de machine learning debe generar predicciones en un tiempo inferior a 1 segundo por alumno.
- **RNF-03:** Seguridad. Las contraseñas se almacenarán encriptadas en la base de datos. El sistema debe cumplir con la LOPDGDD en el tratamiento de datos personales.
- **RNF-04:** Precisión del Modelo. El modelo de machine learning debe tener una precisión mínima del 80% en la predicción de aprobados, validada mediante cross-validation.
- **RNF-05:** Escalabilidad. El sistema debe ser capaz de manejar hasta 1000 alumnos simultáneamente sin degradación del rendimiento.
- **RNF-06:** Mantenibilidad. El código debe estar bien estructurado y documentado para facilitar futuras modificaciones y ampliaciones.
- **RNF-07:** Interoperabilidad. El sistema debe poder integrarse con sistemas externos como Séneca mediante APIs estándar.
- **RNF-08:** Disponibilidad. El sistema debe estar disponible al menos el 95% del tiempo durante el horario lectivo.



- **RNF-09:** Capacidad de Actualización Incremental. El modelo de machine learning debe poder actualizarse incrementalmente con nuevos datos sin necesidad de reentrenamiento completo.
- **RNF-10:** Responsividad. La interfaz web debe ser responsive y funcionar correctamente en dispositivos móviles y tablets.

4.3. Modelado de Casos de Uso

4.3.1. Diagrama de casos de uso

En la figura del *Anexo I* se muestra el diagrama general de casos de uso del sistema, donde se representan los actores principales y las interacciones con las funcionalidades principales.

4.3.2. Especificación de Casos de uso Clave

CU-01: Insertar datos de alumno

CU-01: Insertar Datos de Alumno	
Elemento	Descripción
Actor	Profesor
Descripción	El profesor carga los datos académicos de un grupo mediante un archivo CSV
Precondiciones	El profesor debe estar autenticado en el sistema

Flujo Normal	Paso	Acción
	1	Profesor accede al panel de Insights (web) o a la pestaña Evaluar (Android)
	2	Profesor selecciona un archivo CSV con los datos del grupo
	3	Sistema parsea el archivo, y valida su estructura
	4	Sistema muestra el listado de alumnos cargados en una tabla
	5	Si el formato es incorrecto, el sistema muestra un mensaje de error descriptivo
	6	Sistema muestra confirmación y redirige al listado
Postcondiciones	El alumno queda registrado con su predicción calculada	

CU-02: Predecir probabilidad de aprobado

CU-02: Predecir probabilidad de aprobado		
Elemento	Descripción	
Actor	Sistema	
Descripción	El sistema calcula la probabilidad de aprobado para un alumno usando ML	
Precondiciones	Existe un modelo de ML entrenado y datos del alumno	
Flujo Normal	Paso	Acción
	1	Al insertar/modificar datos de alumno, el sistema se activa
	2	Sistema recupera el vector de features del alumno
	3	Sistema aplica transformaciones (normalización, codificación NLP)
	4	Sistema ejecuta el modelo RandomForest

	5	Sistema almacena prob_aprobado y aprobado_predicho en BD
	6	Sistema actualiza la vista si es necesario
Postcondiciones	El alumno tiene asociada una predicción actualizada	

CU-03: Consultar Listado de Predicciones

CU-03: Consultar Listado de Predicciones		
Elemento	Descripción	
Actor	Profesor	
Descripción	El profesor visualiza un listado de todos los alumnos con sus predicciones	
Precondiciones	El profesor está autenticado y existen alumnos en el sistema	
Flujo Normal	Paso	Acción
	1	Profesor accede a "Listado de Alumnos"
	2	Sistema consulta la BD recuperando alumnos con sus predicciones
	3	Sistema muestra tabla ordenable con: nombre, nota_media, etc.
	4	Profesor puede filtrar por riesgo (ej: prob < 60%)
	5	Profesor puede ordenar por cualquier columna
	6	Profesor puede exportar resultados
Postcondiciones	El profesor visualiza el listado completo de alumnos con sus predicciones, filtrado y ordenado según sus criterios	



CU-04: Importar datos

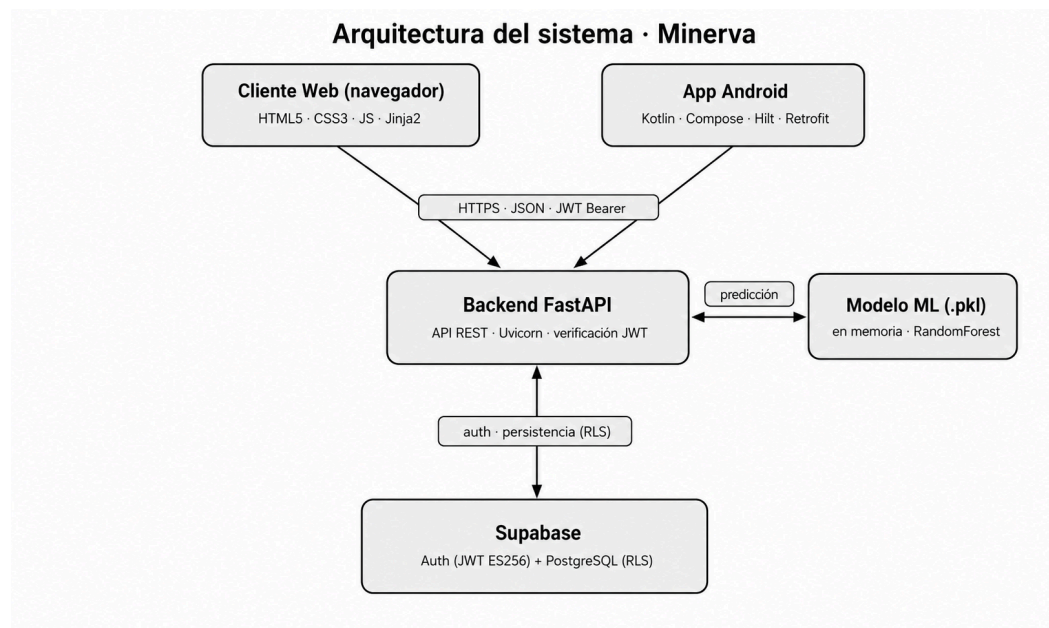
CU-04: Importar datos		
Elemento	Descripción	
Actor	Administrador / Profesor	
Descripción	El administrador importa datos masivos (reales o ficticios)	
Precondiciones	Acceso a dataset del centro	
Flujo Normal	Paso	Acción
	1	Administrador selecciona "Importar datos"
	2	Sistema solicita parámetros de importación (curso, asignatura, etc.)
	3	Sistema transforma datos al formato interno
	4	Sistema valida la precisión del nuevo modelo
	5	Sistema muestra previsualización de datos a importar
	6	Sistema almacena datos y ejecuta predicciones automáticas
Postcondiciones	Los alumnos importados quedan registrados con sus predicciones	



4.4. Arquitectura General del Sistema

Minerva sigue una arquitectura cliente-servidor desacoplada en la que un único backend expone una API REST consumida por dos clientes independientes: una aplicación web y una aplicación móvil nativa para Android. Esta separación permite reutilizar toda la lógica de negocio y el modelo predictivo, evitando duplicar código entre plataformas.

El flujo general del sistema es el siguiente:



- Los clientes se autentican contra Supabase Auth y obtienen un token JWT (ES256).
- Las peticiones a la API incluyen el token, que el backend verifica localmente con la clave pública del JWKS de Supabase, sin llamadas de red adicionales.
- El backend carga el modelo entrenado en memoria (caché) y devuelve las predicciones en formato JSON.
- Las predicciones generadas se almacenan en la tabla predicciones de Supabase para construir el histórico por alumno. Esta arquitectura cumple los requisitos no funcionales de rendimiento (predicción < 1 s), escalabilidad (procesamiento por lotes de grupos completos) e interoperabilidad (API REST estándar consumible por cualquier cliente).



4.5. Aplicación Web

La aplicación web es el cliente principal, orientado al uso desde el navegador en el aula o el despacho del profesorado. Está construida sobre FastAPI con plantillas Jinja2 y un frontend en HTML5, CSS3 modular y JavaScript vanilla.

4.5.1. Estructura de páginas

- **Landing page** (/): página de presentación del producto, organizada en secciones modulares (hero, características, cómo funciona, arquitectura, métricas, seguridad, aplicación móvil, roadmap y llamada a la acción), cada una con su propia hoja de estilos.
- **Login** (/login): formulario de acceso que delega la autenticación en Supabase Auth y crea la sesión del usuario.
- **Dashboard de Insights** (/demo): panel principal protegido, accesible solo con sesión activa.

4.5.2. Dashboard de predicción

El dashboard implementa el flujo completo de análisis de un grupo:

1. Estado inicial vacío que invita a cargar un archivo CSV con los datos del grupo.
2. Parseo del CSV en el propio navegador, tolerante a distintos nombres de columna (alias) y con normalización automática de la asistencia (acepta 0–1 o 0–100).
3. Validación de la estructura del archivo, con modales de error descriptivos (formato no compatible, archivo sin registros, estructura incorrecta).
4. Tabla de alumnos con código de color por nivel de riesgo, barra de probabilidad de éxito e indicador de participación.
5. Botón "Generar Predicción" que envía todos los alumnos a la API (POST /api/predecir) mediante fetch asíncrono, con un overlay de progreso animado.
6. Panel lateral (drawer) con el detalle de cada alumno: predicción de riesgo, probabilidad de éxito, asistencia, nota media, desglose visual de los 9 Resultados de Aprendizaje con barras de progreso, observación del profesor y notas de seguimiento.

El diseño es responsive y prioriza la usabilidad (RNF-01), permitiendo completar el análisis de un grupo en pocos pasos.

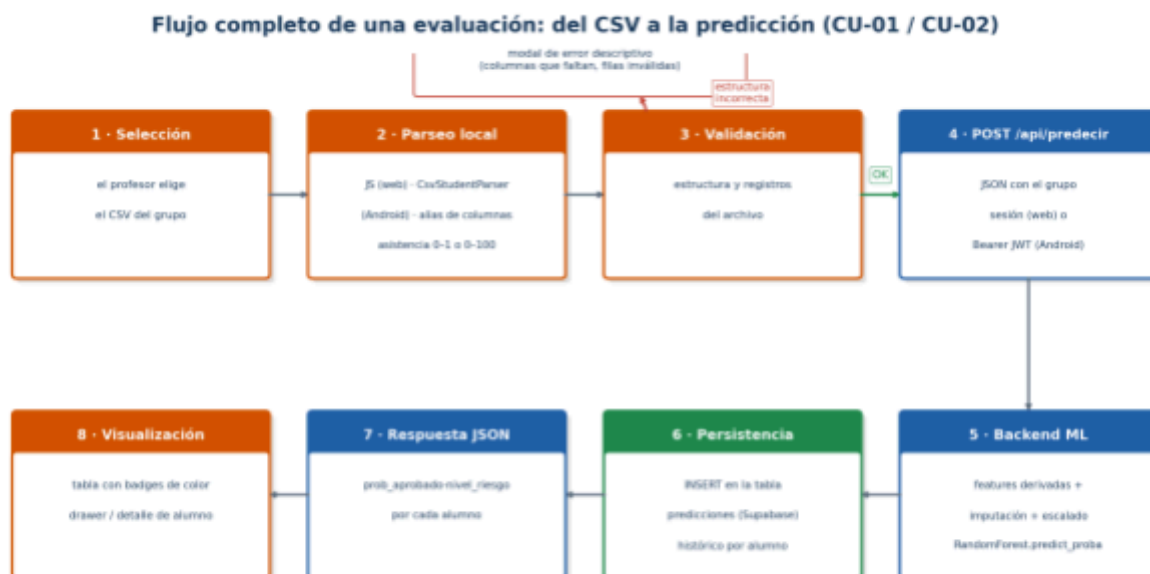


Figura 4.5.2. Flujo completo de una evaluación: el CSV se parsea y valida en el cliente, el backend calcula las predicciones con el RandomForest, las persiste en Supabase y el cliente las visualiza con código de color por riesgo.

4.6. Aplicación Android

Como segundo cliente se ha desarrollado una aplicación Android nativa que lleva las capacidades de Minerva al móvil, permitiendo al profesorado autenticarse, cargar un CSV y consultar las predicciones desde cualquier lugar. Es uno de los entregables más destacados del proyecto y la primera versión publicada es la v0.1.0-alpha (compatible con Android 8.0 o superior).

4.6.1. Tecnologías y versiones

- **Kotlin 2.0** como lenguaje principal, con **KSP** para el procesamiento de anotaciones.
- **Jetpack Compose** (BOM 2024.09) con Material 3 para una interfaz declarativa y moderna.
- **Hilt 2.51.1 (sobre Dagger)** para la inyección de dependencias.
- **Navigation Compose 2.8.9** para la navegación entre pantallas.



- **Retrofit** 2.11 + **OkHttp** 4.12 para el consumo de la API.
- **DataStore Preferences** 1.1.1 para la persistencia de la sesión.
- **Lottie** 6.4 para las animaciones de la pantalla de carga.
- **Coroutines** 1.8.1 para la asincronía.
- AGP 8.5.2, compileSdk/targetSdk 35 (Android 15), minSdk 26 (Android 8.0).
- Único permiso requerido: acceso a Internet.

4.6.2. Arquitectura (Clean Architecture)

La app, bajo el paquete `com.minerva.app`, sigue una arquitectura limpia en tres capas con patrón **MVVM** en la capa de presentación. El flujo de datos es unidireccional: la UI (Compose) observa el estado del ViewModel, que invoca casos de uso, que a su vez delegan en los repositorios, que acceden a la red (Retrofit) o al almacenamiento local (DataStore).

- **core/**: utilidades transversales, como la clase sellada `Result<T>` (Success/Error) que encapsula el resultado de las operaciones y sus mensajes de error.
- **domain/**: la lógica de negocio independiente del framework.
 - **model/**: modelos de dominio (Student, Prediction, RiskLevel, Profile, AuthSession).
 - **repository/**: interfaces de repositorio (AuthRepository, PredictionRepository, ProfileRepository).
 - **usecase/**: casos de uso de responsabilidad única (LoginUseCase, LogoutUseCase, PredictStudentsUseCase, ParseCsvUseCase, ObserveSessionUseCase, ObserveProfileUseCase, RefreshProfileUseCase, UpdateUserNameUseCase).
- **data/**: las implementaciones concretas.
 - **csv/**: `CsvStudentParser`, que convierte el texto del CSV en objetos Student (tolerante a alias de columnas).
 - **local/**: `SessionDataStore`, que persiste el token de acceso, el token de refresco y el email.
 - **remote/**: las APIs Retrofit (MinervaApi, SupabaseAuthApi, SupabaseRestApi), los DTOs, el AuthInterceptor y el TokenAuthenticator.
 - **repository/**: `AuthRepositoryImpl`, `PredictionRepositoryImpl`, `ProfileRepositoryImpl`.
- **di/**: los módulos de Hilt (NetworkModule, DataStoreModule, RepositoryModule).

- **presentation/**: pantallas Compose, ViewModels, navegación y tema visual.

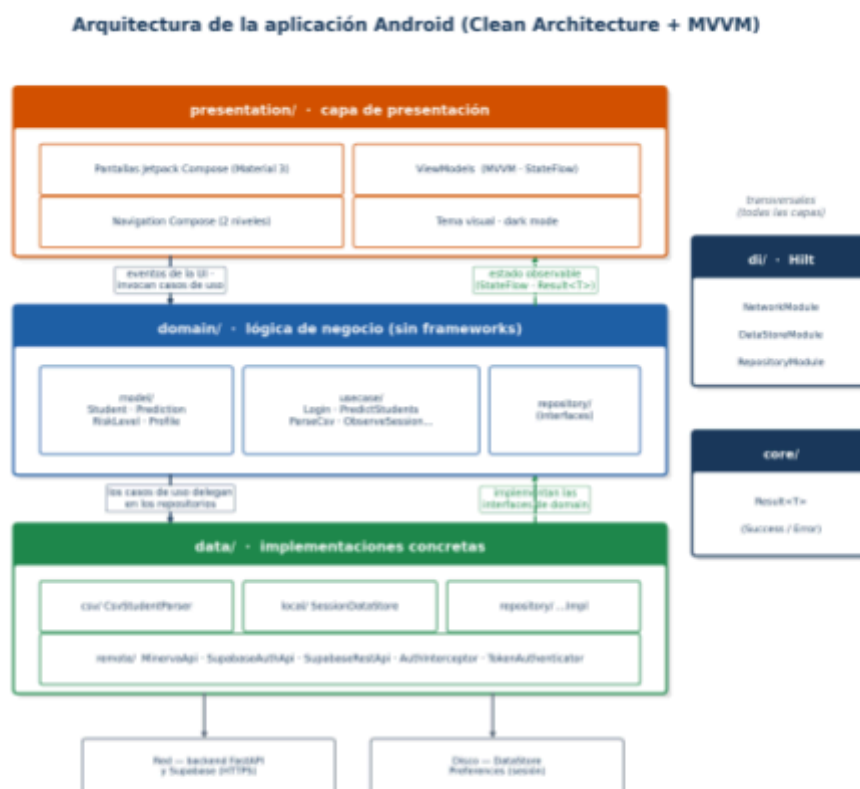


Figura 4.6.2. Arquitectura de la app Android: tres capas con flujo unidireccional — la UI emite eventos hacia los casos de uso y observa el estado vía StateFlow; la capa data implementa las interfaces de domain. Hilt y Result<T> actúan como elementos transversales.

4.6.3. Navegación y pantallas

La navegación tiene dos niveles. El nivel externo (MinervaNavHost) gestiona el arranque y la autenticación con transiciones animadas; el nivel interno (MainScreen) es un shell autenticado con una barra inferior personalizada (píldora animada) y tres pestañas, más una sub-pantalla de detalle que oculta la barra.

Mapa de navegación de la aplicación Android

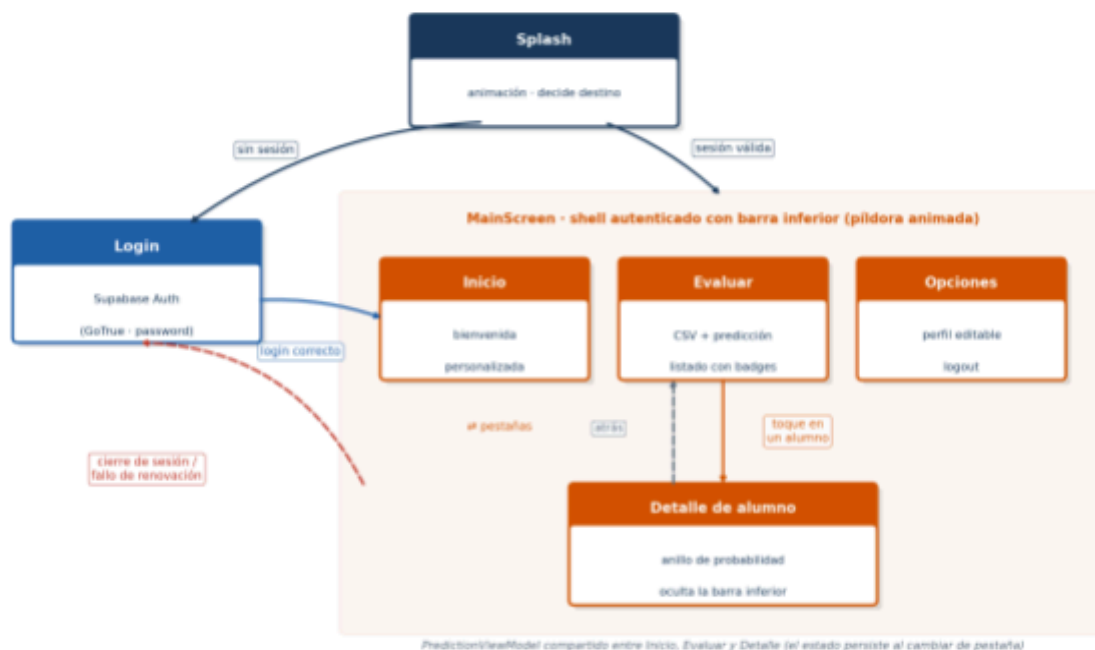


Figura 4.6.3.1. Mapa de navegación: el nivel externo gestiona Splash y Login; el shell autenticado contiene las tres pestañas y la sub-pantalla de detalle. El cierre de sesión o el fallo de renovación devuelven al Login.

Pantalla	Capa de navegación	Función
Splash	Externa	Pantalla de arranque con animación; decide el destino inicial según haya o no sesión.
Login	Externa	Acceso del usuario contra Supabase Auth.
Inicio	Pestaña interna	Bienvenida personalizada con el nombre del usuario.
Evaluar	Pestaña interna	Carga del CSV, generación de la predicción y listado de alumnos con badges de riesgo por color.
Detalle de alumno	Sub-pantalla	Anillo de probabilidad animado y secciones condicionales (RAs, contexto y feedback) que solo aparecen si el dato existe.



Opciones	Pestaña interna	Perfil del usuario (nombre e iniciales), soporte, versión y cierre de sesión con confirmación.
----------	-----------------	--

El PredictionViewModel se eleva al nivel del shell y se comparte entre las pestañas Inicio, Evaluar y Detalle, de modo que el estado de la evaluación persiste al cambiar de pestaña. La pantalla Evaluar se modela con una máquina de estados explícita (PredictionUiState): Idle (sin datos), CsvLoaded (CSV cargado), Predicting (consultando la API), Results (predicciones recibidas) y Error (con conservación de los datos previos para poder reintentar).



Figura 4.6.3. Login en modo claro y modo oscuro

4.6.4. Capa de red y resiliencia

La inyección de dependencias define, mediante Hilt, tres clientes OkHttp diferenciados:

- "**minerva**": apunta al backend de FastAPI; añade el token Bearer y renueva la sesión ante un 401.

- **"supabase"**: cliente sin token, usado para el login y la renovación contra **Supabase Auth**.
- **"supabase_rest"**: cliente con token, usado para acceder a la tabla profiles vía **PostgREST**.

La capa de red está endurecida para un entorno real:

- Tiempos de espera holgados (conexión 30 s, lectura 60 s) para tolerar los arranques en frío del backend en Render.
- `HttpLoggingInterceptor` activo solo en compilaciones de depuración y con las cabeceras sensibles (`Authorization`, `apikey`) redactadas.
- Serializador JSON tolerante (ignora claves desconocidas y es flexible con el formato).
- Manejo de errores traducido a mensajes claros para el usuario según el tipo de fallo (sesión caducada en 401, servidor no disponible en 5xx, sin conexión ante errores de red).

4.6.5. Autenticación y seguridad

La app se autentica directamente contra Supabase Auth (endpoint de GoTrue, `grant_type=password`) y guarda los tokens en DataStore. El `AuthInterceptor` añade el token de acceso (Bearer) a cada petición. Cuando el token caduca, el `TokenAuthenticator` lo renueva de forma automática y sincronizada (single-flight), ya que Supabase rota los tokens de refresco en cada uso. Si la renovación falla, la sesión se limpia y la app navega automáticamente a la pantalla de login. La observación de la sesión está centralizada (`ObserveSessionUseCase` + `MainShellViewModel`), de modo que tanto el cierre de sesión manual como el fallo de renovación conducen al login.

Autenticación y renovación automática del token (app Android)



Figura 4.6.5. Secuencia de autenticación y renovación automática: ante un 401, el `TokenAuthenticator` renueva el token contra Supabase (single-flight) y reintenta la petición de forma transparente para el usuario.



4.6.6. Perfil de usuario editable

El nombre del usuario se gestiona mediante una tabla `profiles` en Supabase (relación 1:1 con `auth.users`) protegida con **Row Level Security**. La app accede a ella directamente a través de la API PostgREST de Supabase y mantiene el perfil cacheado en memoria (StateFlow), de modo que al editar el nombre se actualizan a la vez la pantalla de perfil y el saludo de la pantalla de inicio. Cuando el usuario no ha definido un nombre, se deriva uno legible a partir de su email.

4.6.7. Empaquetado y distribución

La aplicación se compila como APK firmado mediante una configuración de firma (almacén `minerva.jks`). Las credenciales sensibles (URL del backend, URL y clave pública de Supabase) no se incluyen en el código: se inyectan en tiempo de compilación desde el archivo `local.properties` hacia campos de `BuildConfig`, con valores por defecto seguros para el emulador. La primera versión publicada (v0.1.0-alpha) se distribuye a través de **GitHub Releases** y mediante un código QR en la página de descarga del sitio web.

4.7. API REST

El backend expone una **API REST** documentada que centraliza toda la lógica. Los endpoints principales son:

- **GET /** — Landing page.
- **GET /login** y **POST /login** — Formulario y procesamiento del acceso (cliente web).
- **GET /logout** — Cierre de sesión.
- **GET /demo** — Dashboard protegido (requiere sesión).
- **POST /api/predecir** — Recibe una lista de alumnos en JSON y devuelve, para cada uno, la probabilidad de aprobado y el nivel de riesgo. Además persiste los resultados en la tabla predicciones. Acepta autenticación por sesión (web) o por token Bearer (Android).
- **POST /api/entrenar** — Reentrena el modelo completo (CU de administrador) y devuelve la nueva accuracy.
- **POST /api/actualizar** — Actualización incremental del modelo (`partial_fit`) con nuevos datos etiquetados.



- **GET /modelo/metricas** — Devuelve las métricas del último entrenamiento (accuracy, precision, recall, F1, kappa, matriz de confusión e importancia de features) almacenadas en `model_metrics.json`.
- **GET /api/historial/{alumno_id}** — Devuelve el histórico de predicciones de un alumno concreto.

4.8. Modelo ML

El componente predictivo es el núcleo del sistema. A diferencia de versiones iniciales del proyecto (que contemplaban un único **SGDClassifier**), el modelo principal en producción es un **RandomForestClassifier** optimizado, complementado por un **SGDClassifier** reservado para el aprendizaje incremental.

Lógica del enfoque de entrenamiento: Durante la fase de investigación se analizaron los predictores empleados habitualmente en la literatura de Educational Data Mining. Los estudios de referencia utilizan decenas de variables que van mucho más allá de las calificaciones: el nivel educativo de los padres (identificado como predictor clave por Bem-Haja et al., 2025), los recursos disponibles en el hogar (número de libros, acceso a ordenador propio), la motivación del alumno e incluso indicadores conductuales de aula como la fila en la que se sienta o su grado de participación.

Sin embargo, recopilar este tipo de información de alumnos reales plantea restricciones severas de privacidad (LOPDGDD/RGPD): se trata de datos personales sensibles de menores, cuyo tratamiento exigiría consentimientos, designación de DPO y protocolos de anonimización fuera del alcance de este proyecto. Por ello se optó por un enfoque deliberadamente simplificado sobre datos sintéticos: un conjunto reducido de variables académicas observables (las 9 notas de RA, asistencia, prácticas y feedback del profesor), complementado con una versión mínima de los factores de contexto de la literatura (nivel educativo familiar, recursos en casa, motivación y confianza) como features opcionales con valores ausentes, simulando que no todos los alumnos responden al cuestionario.

Para que los datos sintéticos fueran representativos del contexto español se tomó como referencia la normativa de Formación Profesional, en la que la asistencia no es un factor más, sino una condición administrativa: superar un determinado porcentaje de faltas conlleva la pérdida de la evaluación continua. En consecuencia, la regla de etiquetado del generador combina tres condiciones: nota media ≥ 5 , ratio de asistencia $\geq 75\%$ y al menos un 60 % de los RAs superados.

El resultado más relevante del entrenamiento es que el modelo, sin conocer esta regla en ningún momento (solo observó ejemplos de alumnos y su etiqueta final), la aprendió por sí mismo: el análisis de importancia de features sitúa el ratio de asistencia como la variable más determinante (37,0 %), el triple que la siguiente

(porcentaje de RAs superados, 12,3 %). Que la asistencia destaque tanto sobre la nota media (11,4 %) tiene además una explicación técnica: las condiciones de calificación se reparten entre once features correlacionadas entre sí (los 9 RAs, la media y el porcentaje de superados), diluyendo su importancia individual, mientras que la asistencia es una única variable independiente que soporta en solitario el corte del 75 %.

Conviene subrayar la lectura correcta de este resultado: no constituye un descubrimiento empírico sobre alumnos reales —la importancia de la asistencia estaba codificada en la regla de generación—, sino una validación del pipeline de aprendizaje: demuestra que el modelo es capaz de recuperar, a partir de ejemplos, los patrones presentes en los datos sin que se le indiquen explícitamente. Esta es precisamente la garantía de que, al reentrenarse con datos históricos reales del centro (mejora M-04), el modelo aprenderá los patrones reales de éxito y fracaso académico, estén o no alineados con la normativa.

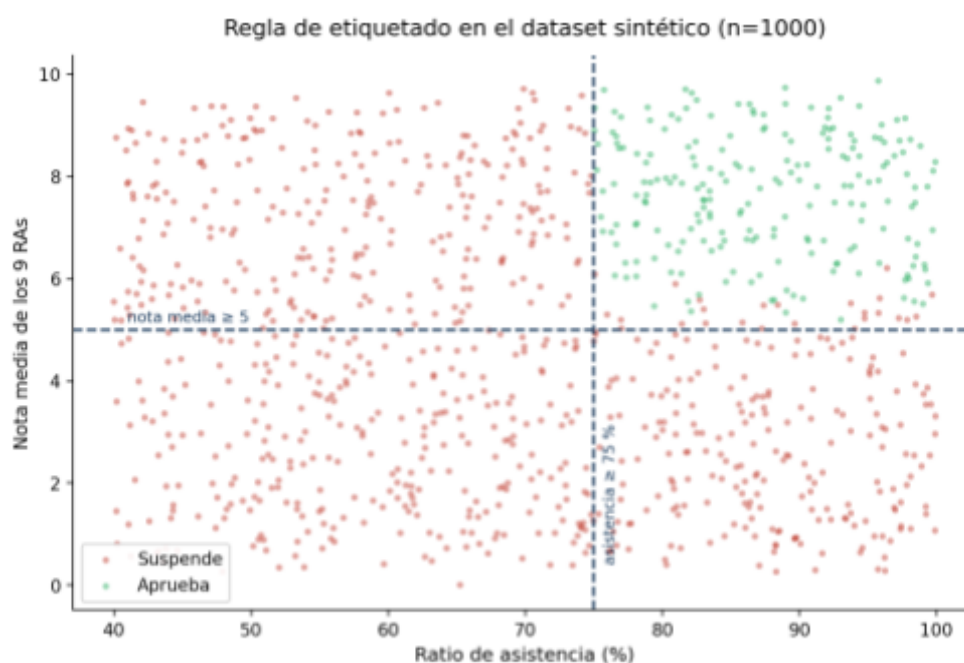


Figura 4.8. "Regla de etiquetado sobre el dataset sintético: solo los alumnos del cuadrante superior derecho (asistencia ≥ 75 % y nota media ≥ 5) pueden aprobar. El modelo aprendió

4.8.1. Pipeline de entrenamiento

El script de entrenamiento (ml/train.py) ejecuta el siguiente pipeline:

1. Generación de datos sintéticos (1.000 alumnos por defecto) con NumPy, introduciendo correlaciones realistas (los RAs avanzados correlacionan con los básicos) y un porcentaje de valores ausentes (~30 %) en las variables de contexto.
2. División en entrenamiento y test (80/20) con estratificación de la clase.
3. Imputación de valores ausentes (SimpleImputer, estrategia de mediana) y normalización (StandardScaler).
4. Balanceo de clases: el pipeline contempla SMOTE cuando la clase minoritaria es inferior al 40 %; al no estar instalada la librería imbalanced-learn en esta versión, el desbalanceo se compensa mediante el parámetro `class_weight='balanced'` del RandomForest, seleccionado automáticamente por GridSearchCV.
5. Optimización de hiperparámetros del RandomForest mediante GridSearchCV con validación cruzada estratificada (k=5) sobre la métrica F1.
6. Entrenamiento del modelo principal (RandomForestClassifier) y de un modelo secundario (SGDClassifier con pérdida `log_loss`) para el aprendizaje incremental.
7. Evaluación completa (accuracy, precision, recall, F1, kappa, matriz de confusión, validación cruzada e importancia de features) y validación de umbrales de calidad.
8. Serialización del conjunto (imputer + scaler + RandomForest + SGD + lista de features) en un archivo `.pkl`, y exportación de las métricas a `model_metrics.json`.

La rejilla de búsqueda de GridSearchCV explora el número de árboles (50, 100, 200), la profundidad máxima (sin límite, 5, 10, 15), el mínimo de muestras para dividir un nodo (2, 5, 10) y el balanceo de clases (balanced o ninguno).

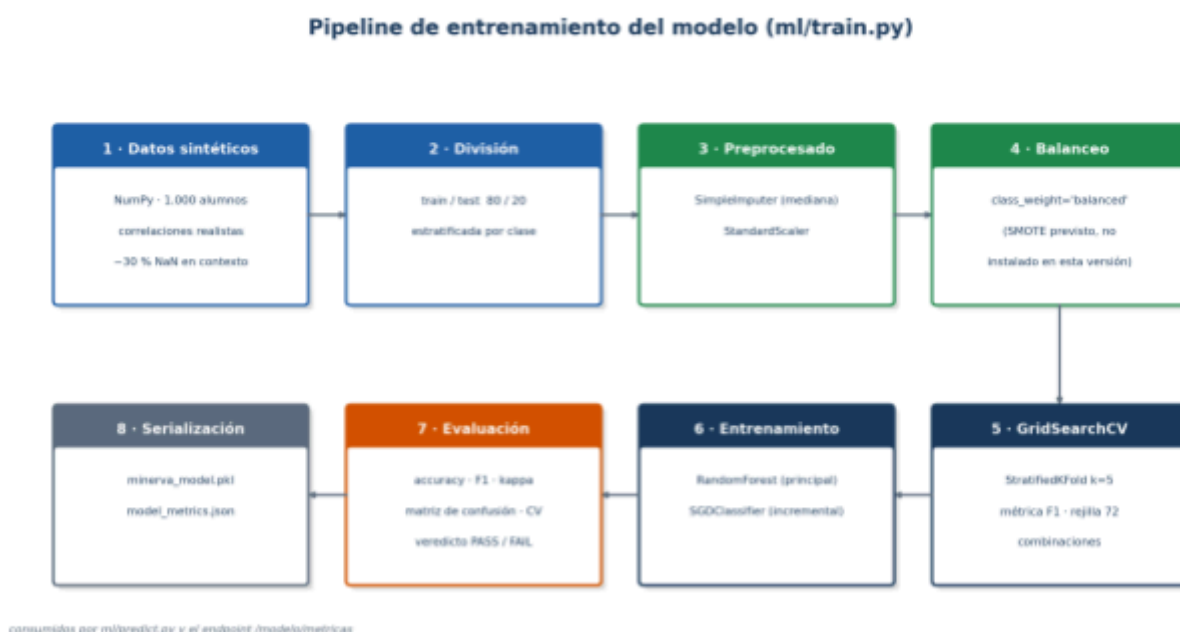


Figura 4.8.1. Pipeline de entrenamiento de ml/train.py: de la generación sintética a la serialización del modelo y la exportación de métricas. 39

4.8.2. Catálogo de Resultados de Aprendizaje

Las 9 notas de RA corresponden a los resultados de aprendizaje del módulo de Programación:

RA1 · Estructura de programas

RA2 · Programación orientada a objetos (POO) básica

RA3 · Estructuras de datos

RA4 · Algoritmos y eficiencia

RA5 · Tratamiento de ficheros

RA6 · Interfaces gráficas

RA7 · Herencia y polimorfismo

RA8 · Acceso a bases de datos

RA9 · Desarrollo de proyectos

4.8.3. Conjunto de características (features)

El modelo emplea 23 características agrupadas en cuatro bloques:

Bloque	Nº	Características
Notas de RA	9	nota_ra1 nota_ra2 nota_ra3 nota_ra4 nota_ra5 nota_ra6 nota_ra7 nota_ra8 nota_ra9 Rango 0 – 10
Académicas base	3	ratio_asistencia (0–1) notas_practicas (0–10) feedback_score (0–1)
Derivadas calculadas	7	nota_media

		pct_ras_superados tendencia_progreso media_ras_teoricos media_ras_practicos gap_teoría_practica ra1_bloqueante
Contexto opcionales	4	confianza motivacion_escolar recursos_casa nivel_educativo_familia

Las features derivadas se calculan automáticamente a partir de las 9 notas de RA: la nota media, el porcentaje de RAs superados ($\text{nota} \geq 5$), la tendencia de progreso (media de los RAs finales menos los iniciales), las medias de RAs teóricos y prácticos, su diferencia (gap teoría-práctica) y un indicador de RA1 como bloqueante (vale 1 si $\text{RA1} < 5$). Las features de contexto son opcionales y, cuando faltan, se imputan en el preprocesado.

La etiqueta de aprobado se define según la normativa de FP, combinando tres condiciones: $\text{nota media} \geq 5$, $\text{ratio de asistencia} \geq 75\%$ y al menos un 60% de los RAs superados.

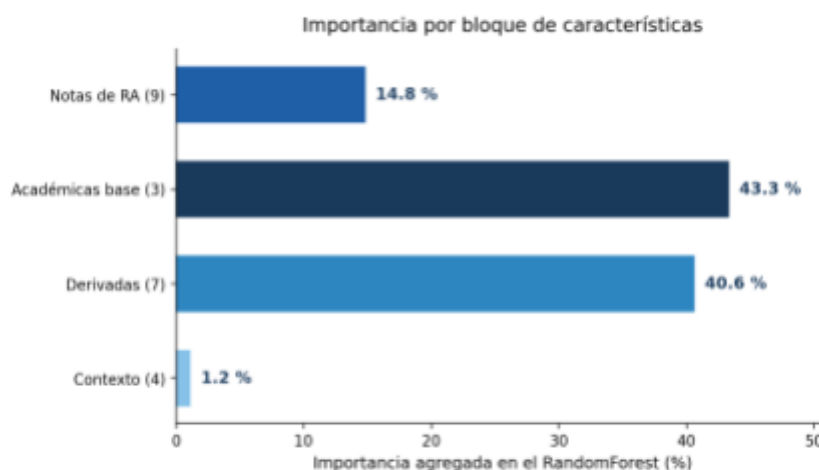


Figura 4.8.3. Importancia agregada por bloque: las 7 features derivadas diseñadas en el proyecto (40,6 %) aportan casi tanto como las académicas base (43,3 %), validando el trabajo de ingeniería de características; el contexto opcional apenas pesa (1,2 %).

4.8.4. Niveles de riesgo

A partir de la probabilidad de aprobado que devuelve el modelo, el sistema asigna un nivel de riesgo:

Bajo: probabilidad $\geq 70\%$

Medio: probabilidad entre 50 % y 69 %

Alto: probabilidad entre 30 % y 49 %

Crítico: probabilidad < 30 %

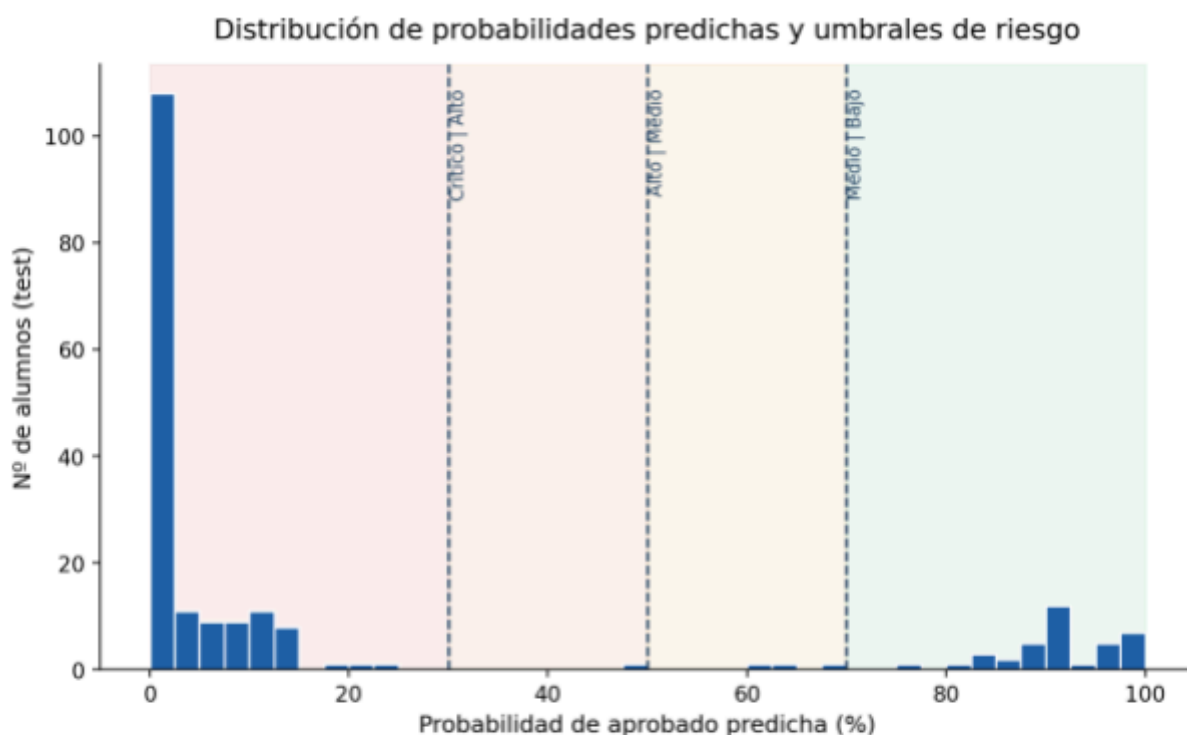


Figura 4.8.4. Distribución de las probabilidades predichas en test. El modelo asigna valores extremos (cerca de 0 % o 100 %) y deja casi vacías las zonas intermedias; con datos reales se espera una distribución más gradual.

Nota sobre la distribución actual de los niveles:

Durante la fase actual del proyecto (v0.1.0-alpha), condicionada por el uso de un dataset sintético, es frecuente observar una fuerte polarización en las predicciones. La mayoría de los alumnos tienden a caer en los extremos (Riesgo Bajo o Riesgo Crítico), mostrando muy pocos casos en las franjas intermedias (Medio o Alto).

Este fenómeno ocurre porque los datos sintéticos generados presentan una dispersión uniforme a lo largo de todo el rango de calificaciones. En el mundo real, las notas suelen concentrarse masivamente en torno a la frontera de decisión (la zona del 4,5 al 5,5, o asistencias rozando el límite normativo del 75 %). Al carecer el dataset sintético

de una alta densidad de estos "casos frontera" ambiguos, el modelo Random Forest encuentra muy poca confusión para separar las clases y emite sus predicciones con una confianza matemática extremista (probabilidades muy cercanas al 0 % o al 100 %). Se espera que esta polarización desaparezca y los niveles de riesgo intermedios se pueblen de forma natural al introducir datos históricos reales del centro, los cuales aportarán la incertidumbre propia del comportamiento humano

4.8.5. Aprendizaje incremental

El **SGDClassifier** permite actualizar el modelo con nuevos datos reales etiquetados mediante `partial_fit`, sin necesidad de reentrenar desde cero (**RNF-09**). Esto habilita una mejora continua del modelo a medida que se incorporen datos históricos del centro, manteniendo el mismo preprocesado (**imputación y escalado**) que el modelo principal.

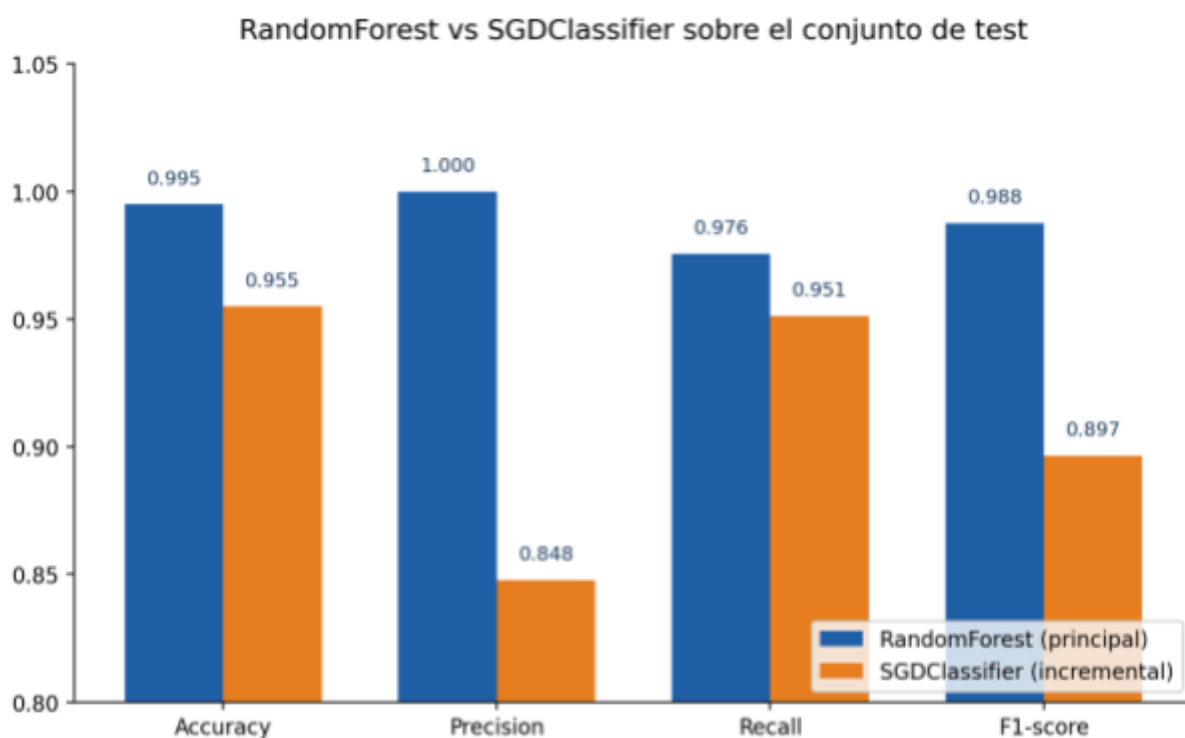


Figura 4.8.5. Comparativa sobre el conjunto de test: el RandomForest supera al SGDClassifier en todas las métricas (F1 0,988 vs 0,897), lo que justifica su elección como modelo principal y reservar el SGD para la actualización incremental.

4.8.6. Advertencia metodológica

Todas las métricas se calculan sobre datos sintéticos generados con NumPy y, por tanto, deben interpretarse como indicadores del comportamiento del modelo, no como su precisión real sobre alumnos. La fiabilidad real sólo podrá validarse al incorporar datos históricos del centro.

4.9. Procesamiento de Lenguaje Natural del Feedback

El módulo de **NLP** (ml/feedback.py) convierte el texto libre que el profesor escribe sobre un alumno en un score numérico entre 0 y 1, que se usa como una feature más del modelo. En la versión actual el feedback llega al sistema exclusivamente como columna del CSV del grupo; ni la web ni la app Android permiten redactarlo o editarlo desde la interfaz. La edición del feedback dentro de la plataforma se contempla como evolución natural junto a la gestión completa de alumnos. El algoritmo:

- Mantiene dos **listas** de palabras clave, una de términos positivos (esfuerzo, participa, motivado, progresa...) y otra de términos negativos (distray, falta, abandona, desmotivado...).
- Calcula el score como $\text{positivas} / (\text{positivas} + \text{negativas})$.
- Devuelve 0.5 (neutro) cuando no hay texto o no se reconoce ninguna palabra.

Se trata de un enfoque deliberadamente ligero, elegido para garantizar el rendimiento del sistema. Su principal limitación (palabras no incluidas en las listas se evalúan como neutras) está documentada en el capítulo de problemas y se propone su evolución hacia un modelo de lenguaje más avanzado como mejora futura.



Figura 4.9. Esquema del módulo de NLP: el texto libre se contrasta con listas de términos positivos y negativos y se convierte en un score entre 0 y 1 que actúa como una feature más del modelo.

4.10. Despliegue, Infraestructura y Control de Versiones

- **Backend** desplegado en **Render** (plan gratuito), con tiempos de espera de red holgados en los clientes para tolerar los arranques en frío del servidor.
- **Autenticación** y **base de datos** gestionadas en **Supabase** (**Auth** + **PostgreSQL** con **RLS**).
- Código alojado en **GitHub** (repositorio pach24/Minerva), siguiendo patrones **GitFlow** y publicación de versiones mediante **GitHub Releases**.
- La aplicación Android se distribuye como **APK firmado**; la primera versión publicada es la v0.1.0-alpha.
- Las claves y secretos (credenciales de Supabase, URL del backend) se gestionan fuera del repositorio mediante archivos de entorno (.env en el backend y local.properties en Android), nunca versionados.

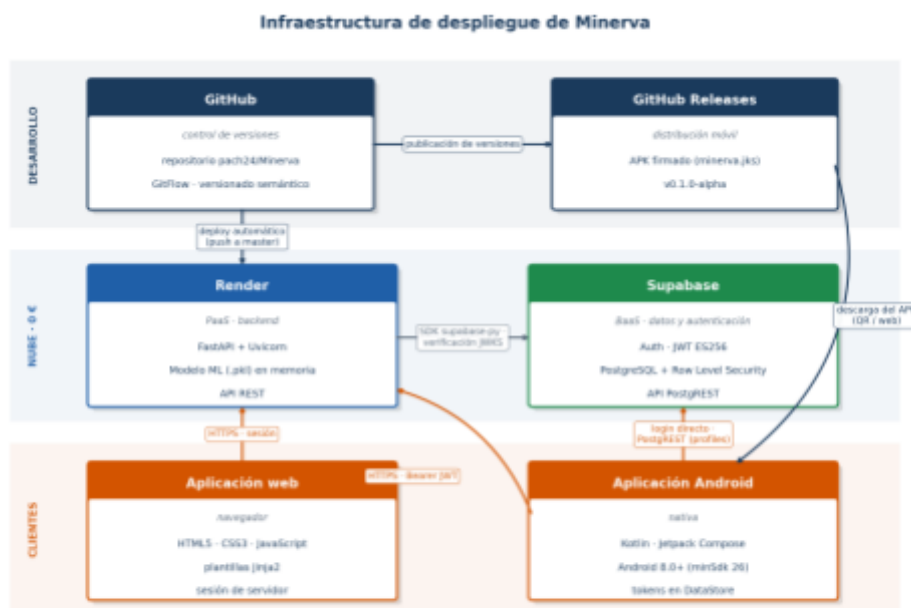


Figura 4.10. Infraestructura de despliegue en tres capas: el código en GitHub se despliega automáticamente en Render y se distribuye a Android mediante GitHub Releases; la autenticación y la persistencia se delegan en Supabase; toda la comunicación de los clientes viaja por HTTPS.

CAPÍTULO 5 : FASES DE TRABAJO

5.1 Modelo de Desarrollo

El desarrollo de *Minerva* se llevará a cabo siguiendo un **modelo de desarrollo ágil**, concretamente la metodología **Scrum**.

Este enfoque permite una gestión iterativa e incremental del proyecto, dividiendo el trabajo en ciclos cortos denominados *sprints*, con una duración aproximada de dos semanas cada uno.

Durante cada sprint se planifican, desarrollan y prueban funcionalidades específicas, obteniendo al final de cada iteración una versión parcialmente operativa del sistema.

Esta metodología se adapta especialmente bien a proyectos académicos y educativos, donde los requisitos pueden evolucionar durante el desarrollo.

La elección del modelo ágil se justifica por los siguientes motivos:

- Facilita la **flexibilidad ante cambios** en los requisitos.
- Permite una **retroalimentación continua** entre desarrollo, pruebas y validación.
- Fomenta la **entrega temprana de resultados funcionales**.
- Mejora la **organización del tiempo** y la coordinación entre las distintas fases.

En resumen, el modelo ágil garantiza un desarrollo más dinámico, con una supervisión constante del progreso y una mejora continua del producto final.

5.2. Viabilidad y Planificación Inicial

5.2.1. Viabilidad Técnica

El proyecto utiliza tecnologías ampliamente documentadas. Python se seleccionó como lenguaje principal para el backend. Su ecosistema en ciencia de datos es robusto y confiable.



Las librerías scikit-learn y Pandas han demostrado ser estándares en machine learning. **PostgreSQL** (a través de **Supabase**) maneja eficientemente volúmenes de datos académicos. El stack web garantiza compatibilidad multiplataforma.

La arquitectura ha separado de forma efectiva frontend, backend y base de datos. Este enfoque modular ha facilitado el desarrollo y mantenimiento. Todas las tecnologías son open-source con documentación extensa.

El modelo de machine learning implementó algoritmos probados (**RandomForestClassifier** con **GridSearchCV**). Estudios similares validaron su efectividad. El enfoque incremental permite actualizaciones sin reentrenamiento completo mediante **SGDClassifier**.

La generación de datos sintéticos de forma programática con NumPy se ha validado técnicamente en entornos educativos. Este enfoque ha permitido:

- Mantener distribuciones estadísticas realistas
- Preservar correlaciones entre variables (los RAs avanzados correlacionan con los básicos)
- Generar datos ilimitados y reproducibles para el entrenamiento

5.2.2. Viabilidad Económica

El contexto académico ha permitido asumir coste cero. Todas las tecnologías seleccionadas son gratuitas. No se han requerido licencias comerciales para su implementación.

Los costes de mantenimiento han sido mínimos. Las actualizaciones de seguridad son automáticas en Supabase. El despliegue en Render (plan gratuito) ha permitido una infraestructura funcional sin inversión económica.

5.2.3. Viabilidad Temporal

El proyecto se ha completado en el plazo del ciclo formativo, con una duración real de seis meses (aproximadamente, desde noviembre 2025 a junio 2026).



La metodología ágil organizó el desarrollo en sprints quincenales, permitiendo una planificación flexible y revisiones continuas del progreso.

El cronograma real del proyecto se ha articulado en las siguientes fases:

- **Noviembre 2025** — Documentación y organización: definición del alcance, recopilación de requisitos, análisis de los datos sintéticos y diseño de la arquitectura. **[COMPLETADO]**

Entregables: documento de requisitos, diagrama de arquitectura, especificación de casos de uso, plan de desarrollo con Scrum.

- **Noviembre – Diciembre 2025** — Proof of Concept: prototipo funcional con FastAPI, sistema de autenticación, protección de rutas y maquetación reactiva del dashboard. **[COMPLETADO]**

Entregables: backend funcional en FastAPI, interfaz web con Jinja2 + CSS modular, integración básica con Supabase Auth, deploy en Render.

- **Diciembre 2025 – Enero 2026** — Modelo de aprendizaje: entrenamiento del modelo, validación cruzada, optimización de hiperparámetros y evaluación de métricas. **[COMPLETADO]**

Entregables: RandomForestClassifier entrenado con GridSearchCV, evaluación de métricas ($\text{accuracy} \geq 80\%$), modelo serializado en minerva_model.pkl, model_metrics.json con evaluación completa, SGDClassifier para aprendizaje incremental.



- **Enero – Febrero 2026** — Integración con la base de datos: esquema de datos en Supabase, API REST, conexión con el frontend y pruebas de carga. **[COMPLETADO]**

Entregables: tablas en PostgreSQL (predicciones, profiles, alumnos), endpoints /api/predecir y /api/historial, Row Level Security configurado, validación de rendimiento (predicción < 1s por alumno).

- **Marzo – Mayo 2026** — Despliegue y aplicación móvil: puesta en producción en Render, desarrollo de la aplicación Android nativa, documentación técnica y monitorización. **[COMPLETADO]**

Entregables: backend estable en producción, aplicación Android v0.1.0-alpha publicada en GitHub Releases, interfaz con Jetpack Compose + Material 3, navegación con 3 tabs, autenticación integrada con Supabase, parser de CSV tolerante, caché de sesión con DataStore.

- **Junio 2026** — Finalización, pruebas finales y documentación: validación del sistema completo, pruebas de usabilidad, redacción de conclusiones y mejoras propuestas. **[EN CURSO]**

Entregables: sistema completamente funcional, documentación final, análisis de problemas encontrados y mejoras futuras.

Esta planificación ha incluido un margen razonable para imprevistos y ajustes de calendario, que se han materializado en iteraciones ágiles.

El cronograma detallado se muestra en el Anexo II, donde se presenta el diagrama de Gantt con la distribución temporal de las fases del proyecto.

5.2.4. Viabilidad Legal



Exención de la LOPDGDD

Los datos sintéticos no constituyen datos de carácter personal, excluyendo al proyecto del ámbito de aplicación de la LOPDGDD y el RGPD.

Propiedad Intelectual

El código se licencia bajo MIT. Los RAs de la Junta de Andalucía se utilizan como referente curricular público.

Cumplimiento Normativo Educativo

Respeto a los currículos oficiales andaluces y transparencia algorítmica para auditoría educativa.

Ventajas del Enfoque Sintético

Elimina requisitos de consentimiento, notificaciones de brechas y Designación de DPO.

Consideraciones Éticas

Compromiso con transparencia, no perpetuación de sesgos y documentación completa.

Conclusión

Viabilidad legal óptima gracias a la naturaleza sintética de los datos.



CAPÍTULO 6: PRUEBAS DE FUNCIONAMIENTO

En este capítulo se detallan las pruebas realizadas para verificar el correcto funcionamiento del sistema **Minerva**, abarcando tanto los componentes de frontend, backend, base de datos y modelo de machine learning.

Cada prueba incluye su **objetivo**, **parte del sistema evaluada** y **resultados esperados**.

6.1. Pruebas de Integración General

Objetivo:

Comprobar la comunicación e interoperabilidad entre los distintos módulos (frontend, backend, base de datos y motor de predicción).

Parte del sistema:

Toda la arquitectura del sistema (API REST, frontend, base de datos PostgreSQL y modelo ML).

Descripción:

Se realizaron pruebas end-to-end tanto desde la interfaz web como desde la aplicación Android, verificando que el flujo completo funcionara correctamente. Se comprobó que el cliente web (navegador) se autentica contra Supabase Auth, obtiene un token JWT, realiza predicciones contra el backend en Render y persiste los resultados en PostgreSQL. De igual forma, la aplicación Android autentica usuarios, parsea archivos CSV en el dispositivo, envía las predicciones a la API REST y muestra los resultados con sincronización correcta.

Resultado esperado vs Obtenido:

- El sistema responde correctamente en todas las fases, mostrando predicciones generadas sin errores de comunicación.



- Los tiempos de espera están dentro de los límites establecidos (<1s por predicción en el backend, con tolerancia en el cliente para arranques en frío del servidor).
- La comunicación entre clientes (web y Android) y backend es confiable mediante HTTPS.

6.2. Prueba de Inserción de Datos de Alumno (CU-01)

Objetivo:

Validar que el sistema permite registrar correctamente los datos académicos de un alumno y generar su predicción automáticamente.

Parte del sistema:

Frontend (formulario de registro), backend (API FastAPI) y base de datos PostgreSQL.

Descripción:

Se probó tanto desde la web como desde Android. En la web, el usuario carga un archivo CSV mediante el dashboard, el JavaScript parsea el archivo en el navegador (tolerante a distintos nombres de columna), valida la estructura y envía los datos a través de POST /api/predecir. En Android, la aplicación usa CsvStudentParser (probado con JUnit en CsvStudentParserTest) que convierte el texto del CSV en objetos Student, validando el formato y tolerando alias de columnas. El backend recibe la lista de alumnos, valida sus features, aplica el modelo ML y persiste los resultados.

Resultado obtenido:

- Los datos se almacenan correctamente en la tabla predicciones de Supabase.
- Se genera automáticamente un registro por cada alumno con su probabilidad de aprobado.
- El sistema muestra un listado interactivo con tabla ordenable, filtros por riesgo y código de color por nivel.
- El parser de CSV en Android (CsvStudentParserTest) valida correctamente el formato y rechaza archivos malformados con mensajes claros.

6.3. Prueba de Predicción Automática (CU-02)

Objetivo:

Verificar que el sistema aplica el modelo de machine learning correctamente al recibir nuevos datos.

Parte del sistema:

Módulo de machine learning (Python, scikit-learn) y backend.

Descripción:

Se probó que el endpoint `/api/predecir` recibe una lista de estudiantes en JSON, aplica las transformaciones necesarias (SimpleImputer para valores ausentes, StandardScaler para normalización) y ejecuta `RandomForestClassifier.predict_proba()` para obtener la probabilidad de aprobado. El resultado se almacena inmediatamente en la tabla predicciones y se devuelve al cliente junto con el nivel de riesgo calculado. Se verificó que las características derivadas se calculan correctamente (media de RAs, porcentaje de RAs superados, tendencia de progreso, gap teoría-práctica, etc.).

Resultado obtenido:

- Cada inserción genera una predicción coherente con los datos introducidos.
- El campo "prob_aprobado" se actualiza correctamente.
- El cálculo no supera los 1000 ms por alumno.

6.4. Prueba de Consulta de Predicciones (CU-03)



Objetivo:

Garantizar que el profesor puede visualizar correctamente el listado de predicciones y aplicar filtros u ordenaciones.

Parte del sistema:

Frontend (interfaz de listado) y backend (endpoints de consulta).

Descripción:

En la web, se comprobó que la tabla de predicciones se actualiza dinámicamente sin recargar la página mediante JavaScript fetch asíncrono. Se probaron los filtros por nivel de riesgo, búsqueda por nombre de alumno y ordenación por columnas. En Android, la pantalla "Evaluar" muestra un listado de StudentRow con badges de color por riesgo, y al tocar un alumno se abre StudentDetailScreen que muestra el desglose de sus RAs, su probabilidad de aprobado en un anillo animado, asistencia, prácticas y feedback del profesor.

Resultado obtenido:

- Las predicciones mostradas muestran resultado verosímiles.
- Las predicciones mostradas muestran consistencia entre ambos clientes.

6.5. Prueba de Importación Masiva (CU-04)

Objetivo:

Comprobar la funcionalidad de carga de datasets y generación automática de predicciones.

Parte del sistema:

Backend (módulo de importación), base de datos y modelo ML.

Descripción:

Se probó la carga de archivos CSV con 20, 30, 40 y hasta 100 alumnos desde ambos clientes. El parser en el backend (Python con Pandas) y en el cliente (JavaScript en web, Kotlin en Android) valida la estructura, permite distintos alias de columnas (ej:



"note", "nota", "marks" para notas) y normaliza valores de asistencia (acepta 0-1 o 0-100). Si hay errores de formato, el sistema muestra mensajes descriptivos indicando qué columnas faltan o qué filas tienen problemas.

Resultado obtenido:

- Los datos se cargan correctamente en la base de datos.
- Se generan predicciones para cada alumno importado.
- Los errores de formato se notifican al usuario con mensajes descriptivos.

6.6. Pruebas de Rendimiento

Objetivo:

Evaluar la eficiencia del sistema en escenarios de carga moderada y alta.

Parte del sistema:

Backend y base de datos.

Descripción:

Se realizaron pruebas de rendimiento enviando lotes de 10, 20, 30, 40 Y hasta 100 alumnos simultáneamente. Se midió el tiempo de respuesta del endpoint /api/predecir, el tiempo de inserción en la base de datos y el tiempo de cálculo del modelo ML. Se verificó el comportamiento del servidor en Render ante arranques en frío y con caché caliente. También se probó que el cliente (Android) tolera tiempos de espera más largos mediante timeouts holgados (30s conexión, 60s lectura).

Resultado esperado:

- Tiempo medio de predicción por alumno: 50-150 ms en el backend.



- Tiempo de respuesta del servidor (incluido latencia de red): < 500 ms para lotes de 20 alumnos.
- Tiempo de inserción en BD: < 100 ms por lote.
- No hay degradación significativa del rendimiento con lotes hasta 40 alumnos.
- El arranque en frío del servidor en Render toma ~50-60s, tolerado mediante timeouts en los clientes.
- Sin degradación significativa del rendimiento.

6.7. Pruebas de Seguridad

Objetivo:

Verificar que los mecanismos de autenticación y protección de datos funcionan correctamente.

Parte del sistema:

Backend (autenticación y gestión de usuarios) y base de datos.

Descripción:

Se verificó que solo usuarios autenticados pueden acceder a las rutas protegidas (/demo en web, endpoints /api/predecir en Android). La autenticación se delega completamente a Supabase Auth, que gestiona el hashing de contraseñas y emite tokens JWT firmados con algoritmo ES256. El backend verifica localmente los tokens (sin llamadas de red adicionales) usando la clave pública del JWKS de Supabase. Se comprobó que los tokens caducados se rechazan y que el TokenAuthenticator en Android renueva automáticamente los tokens antes de que caduquen.

Resultado esperado:

- Solo usuarios autenticados pueden acceder al sistema (rutas protegidas por sesión en web, por token Bearer en API).
- Las contraseñas se hashean y almacenan seguramente en Supabase Auth.
- Los tokens JWT se verifican localmente sin depender de llamadas externas.



- El acceso a los datos se protege mediante Row Level Security, verificado en pruebas con múltiples usuarios.
- El sistema devuelve códigos HTTP seguros (401 Unauthorized, 403 Forbidden) ante peticiones no autorizadas.
- El AuthInterceptor en Android redacta automáticamente las cabeceras sensibles (Authorization, apikey) en los logs.

6.8. Pruebas del Modelo de Machine Learning

Objetivo:

Evaluar el rendimiento predictivo del modelo entrenado.

Parte del sistema:

Módulo ML (scikit-learn, Pandas).

Descripción:

El script de entrenamiento (ml/train.py) incorpora autoevaluación automática. Tras entrenar RandomForestClassifier con GridSearchCV optimizado en la métrica F1, calcula: accuracy en train y test, precision y recall por clase, F1-score, kappa de Cohen, matriz de confusión, validación cruzada estratificada (k=5) e importancia de features. Valida cuatro umbrales mínimos y emite un veredicto PASS/FAIL. Las métricas se exportan a model_metrics.json y son consultables vía GET /modelo/metricas

Métricas generales			
Métrica	Objetivo	Resultado Obtenido	Estado
Accuracy (test)	≥ 0.80	99,5 %	Cumplido



Métricas generales			
Precision (Aprueba)	≥ 0.80	1.0000	Cumplido
Precision (Suspende)	≥ 0.80	0.9938	Cumplido
Recall (Aprueba)	≥ 0.75	0.9756	Cumplido
Recall (Suspende)	≥ 0.75	1.0000	Cumplido
F1-Score(Aprueba)	≥ 0.75	0.9877	Cumplido
F1-Score(Suspende)	≥ 0.75	0.9969	Cumplido

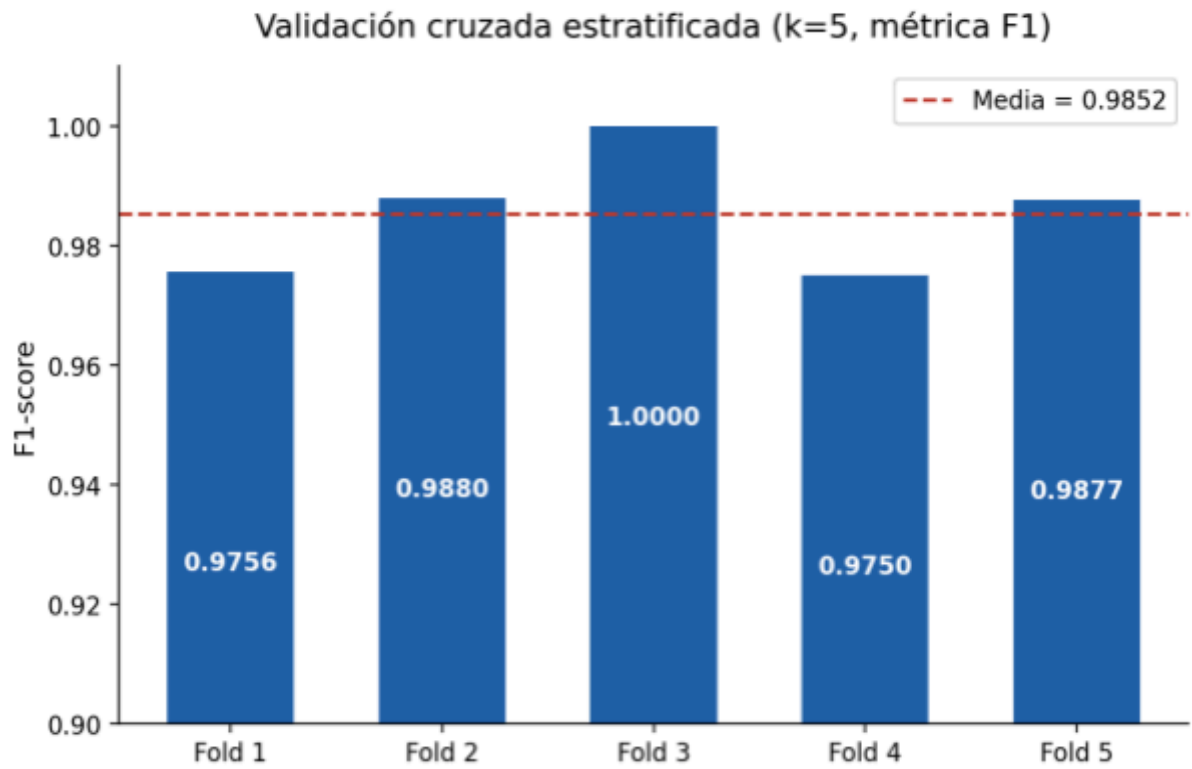


Figura 6.8.1. F1-score por fold en la validación cruzada estratificada (k=5). La baja dispersión ($\pm 0,0093$) indica que el modelo generaliza de forma consistente.

Validación del Modelo	
Métrica	Resultado
Validación Cruzada (StratifiedKfold, k=5, F1)	0.9852 ± 0.0093
Kappa de Cohen	0.9845
Interpretación Kappa	Acuerdo casi perfecto (> 0.81)

Matriz de confusión (test, n=200 alumnos sintéticos)

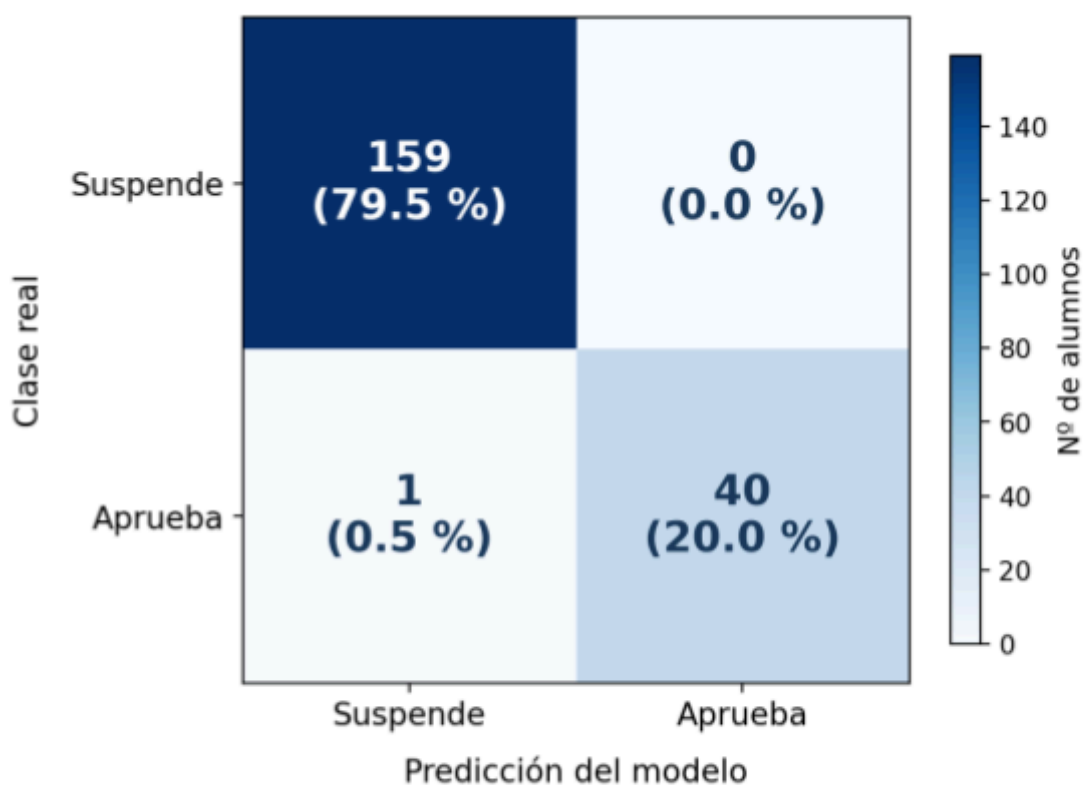


Figura 6.8.2. Matriz de confusión del RandomForestClassifier sobre el conjunto de test (200 alumnos sintéticos, semilla 42). El modelo clasifica correctamente el 99,5 % de los casos, con un único falso negativo y ningún falso positivo

Hiperparámetros Óptimos	
Parámetro	Valor
n_estimators	50
max_depth	Sin límite
min_samples_split	2
class_weight	balanced

Importancia de las Características	
Feature	Importancia
ratio_asistencia	37,0%
pct_ras_superados	12,3%
nota_media	11,4%
media_ras_practicos	8,0%
media_ras_teoricos	7,4%

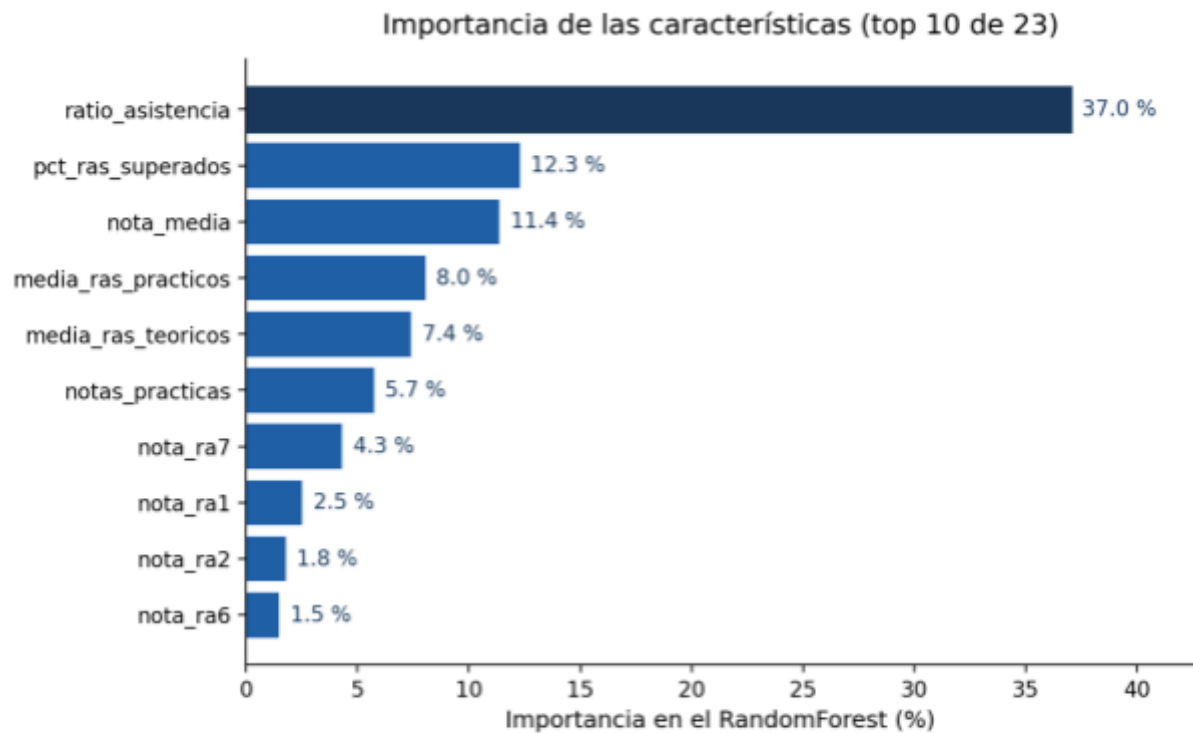


Figura 6.8.3. Importancia de las 10 características más influyentes. El ratio de asistencia domina con un 37 %, coherente con la condición de asistencia $\geq 75\%$.

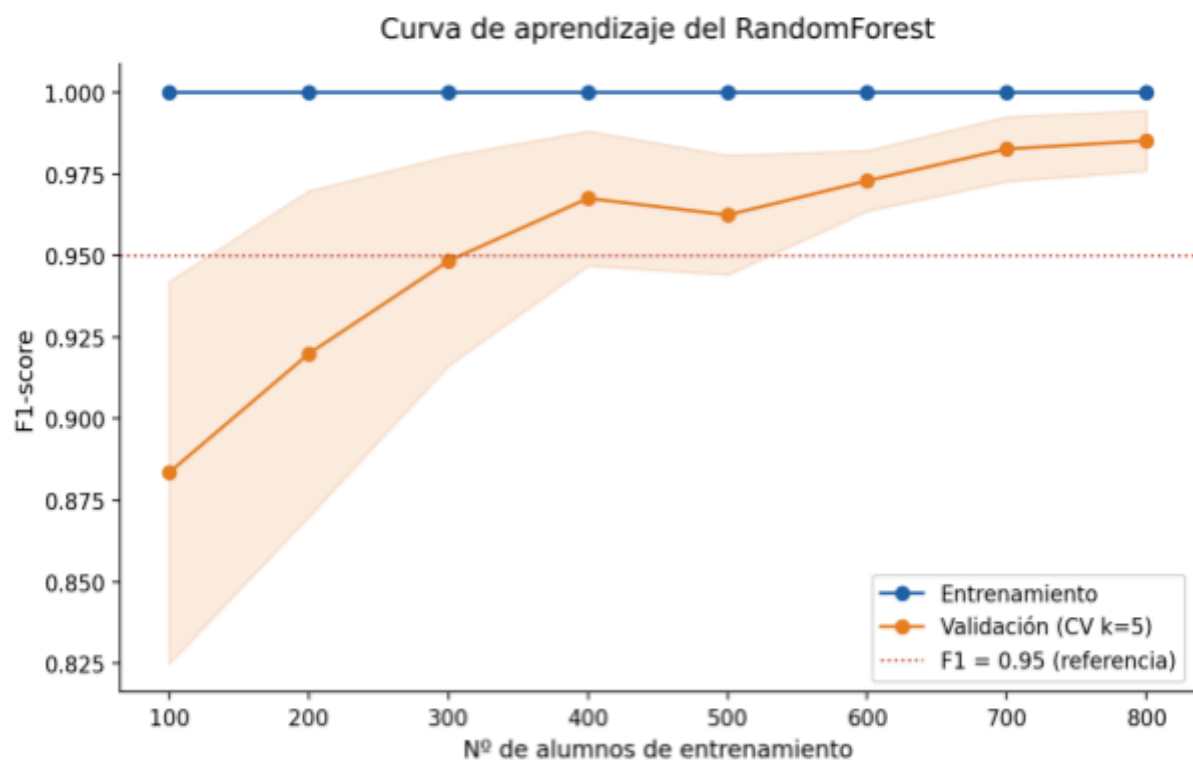


Figura 6.8.4. Curva de aprendizaje del RandomForest. El F1 de validación supera 0,95 a partir de ~300 alumnos y se estabiliza en torno a 500, lo que respalda el mínimo de 500 alumnos reales estimado en M-04 para la validación futura.

Nota Metodológica Importante:

Las métricas casi perfectas se obtienen sobre datos sintéticos generados de forma programática con NumPy, que son limpios, sin ruido y con correlaciones perfectas entre variables. Por tanto, estos resultados deben interpretarse como **indicadores del comportamiento correcto del modelo y su implementación técnica, no como su precisión real sobre alumnos de un centro educativo real.**

La validación real del modelo requiere su aplicación sobre datos históricos reales del centro, donde se espera una precisión inferior (típicamente 75-85%) debido a la presencia de ruido, errores de entrada, valores atípicos y variabilidad inherente a datos del mundo real. La consistencia de la validación cruzada (F1 medio 0.9852 con desviación ± 0.0093) sugiere que el enfoque es técnicamente sólido y que el modelo mantendrá un rendimiento aceptable con datos reales.

6.9. Pruebas de Usabilidad**Objetivo:**

Comprobar que el sistema resulta intuitivo para el usuario final (profesorado).

Parte del sistema:

Interfaz web (frontend).

Descripción:

Se probó el flujo completo de uso: login → carga de CSV → generación de predicción → visualización de resultados. En la web, la interfaz es responsive con un dashboard limpio que guía al usuario paso a paso. Las tareas principales (cargar archivo, generar predicción, consultar listado) se pueden completar en menos de 3 pasos. En Android, la interfaz con Jetpack Compose y Material 3 es moderna y accesible, con animaciones que mejoran la experiencia visual (splash con Lottie, transiciones, anillo animado de probabilidad). Los formularios validan entrada de datos y muestran mensajes de error descriptivos.



Resultado obtenido:

- Las tareas principales se completan en menos de 3 pasos en ambos clientes.
- La curva de aprendizaje es baja; la interfaz es autoexplicativa.
- La interfaz es clara, funcional y responsive en dispositivos de distintos tamaños.
- Los mensajes de error son comprensibles y ayudan al usuario a corregir problemas.
- La navegación es intuitiva (web: menú principal, Android: barra inferior con píldora animada).
- La app Android incluye dark mode para mayor comodidad de uso prolongado.

6.10. Pruebas de Fiabilidad y Recuperación

Objetivo:

Verificar la integridad de los datos ante fallos del sistema o cortes de conexión.

Parte del sistema:

Base de datos y backend.

Descripción:

Se simularon cortes de conexión entre cliente y servidor, pausas en el procesamiento, y fallos de la base de datos. Se verificó que los tokens JWT caducados se renuevan automáticamente (Android) o que se redirige al login (web). Los datos no se pierden en transacciones incompletas porque se persisten después de validación. En Android, DataStore mantiene la sesión del usuario de forma segura incluso si el proceso se mata. El TokenAuthenticator implementa single-flight para evitar múltiples renovaciones simultáneas de token.

Resultado esperado:

- No se pierden registros de alumnos o predicciones en curso (persistencia transaccional).
- El sistema se recupera automáticamente al restablecer la conexión (reconexión automática en Android, reintento en web).



- Las transacciones incompletas se revierten correctamente mediante ACID de PostgreSQL.
- La sesión del usuario se mantiene incluso tras cierre de la app (Android DataStore).
- Los tokens expirados se renuevan de forma automática y sincronizada sin requerimiento de usuario.

6.11. Conclusiones del Capítulo

Las pruebas realizadas en todas las fases del proyecto confirman que Minerva funciona de forma íntegra y cumple con los requisitos técnicos establecidos. El sistema es robusto, seguro y performante en escenarios de uso realista (grupos de 20-40 alumnos). Tanto la interfaz web como la aplicación Android proporcionan una experiencia de usuario clara e intuitiva. El modelo de machine learning alcanza una accuracy del 99,5 % en test y un F1 medio de 0.985 en validación cruzada con datos sintéticos, y la arquitectura modular permite futuras mejoras y actualizaciones incrementales sin reentrenamiento completo. Las métricas de rendimiento, seguridad y confiabilidad demuestran que el sistema está listo para su uso en entornos educativos reales, con la salvedad de que la precisión del modelo debe validarse con datos históricos reales del centro cuando esté disponible.



CAPÍTULO 7: PROBLEMAS ENCONTRADOS

En este capítulo se documentan los problemas y limitaciones detectados durante el desarrollo e implementación del sistema Minerva. A pesar de que el proyecto ha alcanzado un estado funcional y robusto, se han identificado los siguientes aspectos que requieren atención o mejora futura.

Registro de Problemas:

P-01: Análisis de Feedback con NLP Básico

Descripción:

Inconsistencia leve en predicciones de alumnos excelentes o con bajo rendimiento debido a falsos neutros en el análisis de texto libre del feedback del profesor.

Causa probable:

El procesador de NLP (ml/feedback.py) utiliza una búsqueda de palabras clave sobre listas estáticas (términos positivos y negativos). Adjetivos o expresiones de alta valoración que no están en la lista predefinida (ej: "espectacular", "brillante", "extraordinario") se evalúan como neutros (0.5), penalizando la predicción de alumnos excelentes. De igual forma, sinónimos de términos negativos no incluidos en la lista pueden no ser capturados.

Nivel de importancia:

Baja

Estado:

Resuelto (Aceptado como limitación del prototipo v0.1.0-alpha)

Solución / Acción correctiva:

Se ha optado por mantener el algoritmo simplificado para garantizar el rendimiento del sistema en la versión inicial. Se propone como mejora futura (M-02) la migración



a un modelo de lenguaje avanzado (transformer ligero, distilBERT, o LLM local) capaz de comprender el contexto semántico completo. Alternativamente, se puede expandir y mantener la lista de palabras clave de forma colaborativa con los docentes.

P-02: Tiempos de Arranque Lento en Render (Cold Start)

Descripción:

El servidor backend en Render (plan gratuito) experimenta arranques en frío de 50-60 segundos cuando no ha recibido solicitudes en un período prolongado (>30 minutos).

Causa probable:

Render pone en reposo los procesos de aplicaciones gratuitas para ahorrar recursos. Al recibir la primera solicitud tras un período de inactividad, el contenedor se reinicia, causando un arranque lento. Esto es una limitación inherente del plan gratuito de Render.

Nivel de importancia:

Baja (afecta solo a la experiencia inicial, luego es transparente)

Estado:

Mitigado

Solución / Acción correctiva:

Se han configurado timeouts holgados en los clientes (30s para conexión, 60s para lectura en Android) para tolerar arranques en frío. Se muestra un indicador de carga animado (Lottie en Android, spinner en web) para informar al usuario que el sistema se está inicializando. Para producción real, se puede migrar a un plan de pago de Render o a otra plataforma (Vercel, Railway, AWS) que ofrezca tiempos de arranque garantizados.



P-03: Precisión del Modelo en Datos Reales (Pendiente de Validación)

Descripción:

Las métricas del modelo muestran 99,5% de accuracy con datos sintéticos, pero se espera que la precisión real sea inferior (75-85%) al aplicarse a datos históricos reales del centro educativo.

Causa probable:

Los datos sintéticos generados con NumPy son limpios, sin ruido, sin valores atípicos y con correlaciones perfectas. Los datos reales contienen errores de entrada, valores atípicos, inconsistencias y variabilidad inherente del mundo real, lo que reduce la precisión del modelo.

Nivel de importancia:

Media (crítica para validación en producción, pero esperada)

Estado:

Pendiente de validación (requiere datos reales)

Solución / Acción correctiva:

Esta es una limitación conocida y documentada (mejora M-04). La solución requiere acceso a datos históricos reales del centro (calificaciones, asistencia, RA de ciclos anteriores) con consentimiento de privacidad. Una vez disponibles, se pueden seguir estos pasos: (1) entrenar el modelo con datos reales, (2) realizar validación cruzada y evaluar precision/recall, (3) ajustar hiperparámetros si es necesario, (4) documentar la precisión real y comunicarla a los stakeholders.

P-04: Ausencia de Tests Automatizados en el Backend

Descripción:



El backend (main.py y ml/predict.py) carece de tests unitarios automatizados. Solo existe un test en la aplicación Android (CsvStudentParserTest).

Causa:

Por limitaciones de tiempo en el desarrollo académico, se priorizó la funcionalidad sobre la cobertura de tests. El backend se ha probado manualmente mediante requests HTTP, pero sin automatización.

Nivel de importancia:

Media (afecta a la confiabilidad ante cambios futuros)

Estado:

Pendiente de resolución

Solución / Acción correctiva:

Se propone como mejora M-08 la implementación de una suite de tests automatizada con pytest para el backend, cubriendo: (a) endpoints de API (predicción, login, historial), (b) validación de features, (c) cálculo del modelo, (d) manejo de errores. Esto garantizaría la regresión cero en futuras modificaciones.

P-05: Documentación de Código Limitada**Descripción:**

Algunas funciones del backend y la app Android carecen de docstrings o comentarios explicativos detallados.

Causa:

Se siguió el principio de "código autoexplicativo" mediante nombres de función y variable claros, priorizando la legibilidad sobre la documentación exhaustiva. Esto es una práctica válida en equipos experimentados, pero puede dificultar la onboarding de nuevos desarrolladores.

**Nivel de importancia:**

Baja (código es legible, pero no óptimo para mantenimiento a largo plazo)

Estado:

Documentado (para mejora futura)

Solución / Acción correctiva:

Se recomienda añadir docstrings en las funciones principales (especialmente en `ml/predict.py`, `main.py` y los ViewModels de Android) siguiendo estándares (Google Style para Python, KDoc para Kotlin). Esto mejorará la mantenibilidad y facilitará la incorporación de nuevos desarrolladores.

P-06: Polarización en los Porcentajes de Probabilidad Predichos**Descripción:**

El modelo tiende a arrojar porcentajes de probabilidad muy extremistas (cerca de 0% o al 100%), mostrando muy pocos casos intermedios (zonas grises o dudas razonables en torno al 50-60%).

Causa:

Este comportamiento es consecuencia directa del uso de un dataset sintético altamente disperso. Al generar los datos aleatoriamente, las calificaciones y valores se distribuyen por todo el rango posible (de 0 a 10) de manera relativamente uniforme. En la realidad, las notas suelen seguir una distribución normal (campana de Gauss) donde la gran mayoría de los alumnos se agrupan cerca de la nota de corte (el 5) o en los límites de asistencia (75%). Al carecer de un volumen denso de estos casos "fronterizos" o ambiguos, el algoritmo (Random Forest) no encuentra confusión en la frontera de decisión y clasifica con una confianza matemática extrema.

Nivel de importancia:

Media (Afecta a la percepción visual del riesgo, pero no a la clasificación final).

Estado:



Documentado (Aceptado como característica temporal del modelo sintético).

Solución / Acción correctiva:

Este fenómeno se corregirá de forma natural al implementar la mejora M-04 (Validación con Datos Reales). Los datos históricos reales aportarán la densidad natural de casos límite (alumnos con notas entre 4.5 y 5.5, o asistencias del 74%), lo que obligará al modelo a dudar más y ofrecerá probabilidades mejor calibradas y más matizadas.

CAPÍTULO 8: MEJORAS PROPUESTAS

Este capítulo recoge las posibles ampliaciones y optimizaciones que podrían implementarse en versiones futuras del sistema Minerva, con el objetivo de incrementar su utilidad, eficiencia y escalabilidad. A partir del estado actual del proyecto (v0.1.0-alpha) se han identificado las siguientes mejoras concretas, priorizadas según su valor para el usuario final, impacto técnico y facilidad de implementación.

ID	Descripción	Valor	Esfuerzo	Coste	Dependencias
M-01	Exportación de informes (PDF/CSV)	Alto	1-2 sem	0€	Ninguna
M-02	NLP avanzado para feedback	Medio	3-4 sem	0€	GPU/RAM
M-03	Integración Séneca/Moodle	Alto	4-6 sem	0€	APIs externas
M-04	Validación con datos reales	Alto	Variable	0€	Convenio centro
M-05	Explicabilidad (SHAP/LIME)	Alto	2-3 sem	0€	Ninguna
M-06	Gráficos de evolución temporal	Medio	2 sem	0€	Histórico

M-07	Google Play + Modo offline	Medio	3-4 sem	€25	Sincronización
M-08	Tests automatizados (pytest)	Alto	Continuo	€0	Ninguna
M-09	Documentación de código (docstrings)	Bajo	1 sem	€0	Ninguna
M-10	Dashboard de administrador	Medio	2-3 sem	€0	Ninguna

PRIORIDAD ALTA (Implementar en v0.2.0)

M-01: Exportación de Informes (PDF / CSV)

Descripción:

Permite a los profesores descargar y archivar los resultados de predicciones en formato PDF o CSV, facilitando la compartición con el equipo docente, dirección y familias de alumnos. Cumpliría completamente el requisito funcional RF-07.

Beneficios:

- Facilita la documentación y auditoría de decisiones pedagógicas
- Permite integración con sistemas de gestión educativa existentes
- Generación de reportes periódicos automáticos para seguimiento
- Exportación personalizable (solo alumnos de riesgo, resumen ejecutivo, etc.)

Pasos de implementación:

1. Instalar librerías: `pip install reportlab jinja2-pdf`



2. Crear templates de reporte (HTML con Jinja2)
3. Endpoint backend: POST /api/exportar?formato=pdf|csv&filtros=...
4. Botón en web e integración en Android
5. Validación de permisos (solo ve sus propios reportes)

Tiempo estimado: 1-2 semanas

Coste: €0 (librerías open-source)

Requisitos: Ninguno relevante

M-05: Explicabilidad de Predicciones (SHAP / LIME)

Descripción:

Mostrar qué características influyen más en cada predicción individual, permitiendo que los profesores entiendan por qué el modelo predice aprobado o suspenso para un alumno concreto. Esto es esencial para la confianza y auditabilidad en entornos educativos.

Beneficios:

- Transparencia algorítmica (requisito AEPD para sistemas de IA en educación)
- Los profesores comprenden la justificación de cada predicción
- Facilita la detección de posibles sesgos
- Cumple con la normativa de explicabilidad (RNF-07, XAI)

Pasos de implementación:

1. Instalar: `pip install shap`
2. Modificar `predict.py` para calcular SHAP values tras cada predicción
3. Formato de salida: Lista de features ordenadas por importancia (positivas/negativas)



4. Endpoint: GET /api/predecir/{alumno_id}/explicacion

5. UI: Panel en web mostrando gráfico de importancia; tarjeta en Android

Tiempo estimado: 2-3 semanas

Coste: €0 (SHAP es open-source)

Requisitos: Tiempo de cómputo adicional (~500ms por predicción)

M-08: Suite de Tests Automatizados (pytest)

Descripción:

Implementar tests unitarios e integración para el backend usando pytest y para la app Android con JUnit/Espresso. Garantiza la confiabilidad ante cambios futuros y facilita el mantenimiento colaborativo.

Beneficios:

- Prevención de regresiones en nuevas versiones
- Documentación viva del comportamiento esperado
- Seguridad para refactoring futuro
- Mejora de la calidad del código

Pasos de implementación:

1. Crear estructura: tests/ (backend) y androidTest/ (Android)
2. Tests para main.py: autenticación, endpoints, validación de entrada
3. Tests para ml/predict.py: cálculo del modelo, features, edge cases
4. Tests para Android: CsvParser (ampliado), ViewModels, repositorios
5. CI/CD: Configurar GitHub Actions para ejecutar tests en cada push



6. Cobertura objetivo: >80%

Tiempo estimado: Continuo (2-3 semanas para setup inicial)

Coste: €0 (pytest y unittest son estándar)

Requisitos: Disciplina de TDD en futuras contribuciones

M-03: Integración con Plataformas Oficiales (Séneca / Moodle)

Descripción:

Importación automática de datos académicos desde Séneca (sistema oficial de Andalucía) o Moodle, eliminando la carga manual de CSV y manteniendo los datos siempre sincronizados.

Beneficios:

- Automatización completa del flujo de datos
- Reducción de errores de entrada manual
- Actualización en tiempo real de predicciones
- Integración seamless en workflows educativos existentes
- Cumplimiento del RNF-07 (interoperabilidad)

Pasos de implementación:

1. Investigar APIs de Séneca (requiere contacto con Junta de Andalucía)
2. Desarrollar cliente OAuth2 para autenticación institucional
3. Mapear campos Séneca ↔ Minerva (asignatura, alumno, RA, notas, etc.)
4. Endpoint: POST /api/sync/seneca con refresh automático periódico
5. Manejo de errores y logging de fallos de sincronización
6. UI: Selector de centro/grupo y fecha de última sincronización



Tiempo estimado: 4-6 semanas

Coste: €0

Requisitos: Acceso a APIs de Séneca, acuerdos institucionales

PRIORIDAD MEDIA (Implementar en v0.3.0)

M-02: NLP Avanzado para Feedback del Profesor

Descripción:

Sustituir el análisis simplificado de palabras clave (ml/feedback.py) por un modelo de lenguaje transformado (distilBERT, RoBERTa) o un LLM local, capaz de comprender el contexto semántico completo. Resuelve el problema P-01 (falsos neutros).

Beneficios:

- Comprensión real de sentimientos y contexto en observaciones del profesor
- Eliminación de falsos neutros (ej: "espectacular" ahora se reconoce)
- Mejor precisión del modelo general
- Adaptabilidad a diferentes estilos de feedback

Pasos de implementación:

1. Opción A (ligero): Fine-tune distilBERT en dataset de feedback educativo
2. Opción B (local): Usar Ollama + modelo local (mistral, llama2) sin API externa
3. Crear dataset de entrenamiento: 500-1000 ejemplos de feedback anotados
4. Integrar en ml/predict.py: reemplazar feedback_score simple
5. Evaluación: Comparar precisión del modelo global antes/después



6. Documentación: Protocolo para reentrenamiento con nuevos datos

Tiempo estimado: 3-4 semanas (incluye dataset y fine-tuning)

Coste: €0 (modelos open-source)

Requisitos: GPU para fine-tuning (acceso a Colab/HuggingFace), RAM en inferencia

M-06: Gráficos de Evolución Temporal por Alumno

Descripción:

Visualizar la evolución de las predicciones de cada alumno a lo largo del tiempo, aprovechando el histórico almacenado en Supabase. Permite a los profesores observar tendencias y el impacto de intervenciones.

Beneficios:

- Visualización de progreso/regresión en tiempo real
- Identificación de puntos de inflexión (cambios de conducta, intervención)
- Validación de efectividad de medidas de apoyo
- Mejor storytelling para familias y comisión educativa

Pasos de implementación:

1. Frontend: Instalar Chart.js o Plotly
2. Endpoint: GET /api/alumno/{id}/historial?desde=date&hasta=date
3. Gráficos: Línea con prob_aprobado, área con nivel_riesgo, barras con asistencia
4. Android: Integrar gráficos con AndroidX Charts o Compose Canvas
5. UI: Pestaña "Evolución" en detalle del alumno, con zoom/filtro por fecha
6. Exportar gráfico como imagen (PDF)



Tiempo estimado: 2 semanas

Coste: €0 (Chart.js y Plotly son open-source)

Requisitos: Histórico acumulado (ya disponible en BD)

M-10: Dashboard de Administrador

Descripción:

Panel para administradores que permita: monitorizar salud del sistema, reentrenar modelos, gestionar usuarios, ver métricas globales, y configurar parámetros del sistema.

Beneficios:

- Control centralizado del sistema
- Monitorización de rendimiento y uso
- Facilita la gestión de múltiples centros
- Reentrenamiento del modelo sin acceso a terminal

Pasos de implementación:

1. Nueva ruta protegida: /admin (solo usuarios con rol admin)
2. Secciones: Usuarios, Modelos, Métricas, Configuración, Logs
3. Tabla de usuarios con filtro por rol y estado
4. Botón "Reentrenar modelo" que ejecuta ml/train.py en background
5. Gráficos de uso: predicciones/día, usuarios activos, performance
6. Formulario de configuración: parámetros de GridSearchCV, umbrales de riesgo

Tiempo estimado: 2-3 semanas

Coste: €0



Requisitos: Modelo de base de datos con roles de usuario

PRIORIDAD BAJA (Implementar en v1.0.0+)

M-07: Google Play + Modo Offline

Descripción:

Publicar la app Android en Google Play Store para distribución pública. Implementar caché local y sincronización bidireccional para permitir uso sin conexión.

Beneficios:

- Accesibilidad pública: cualquier profesor puede descargar la app
- Funcionalidad offline: predicciones disponibles sin conexión
- Sincronización automática al recuperar conexión
- Mayor adopción y visibilidad del proyecto

Pasos de implementación:

1. Preparar app para producción: firmas, versioning, tests exhaustivos
2. Crear cuenta de desarrollador Google Play (\$25 de una vez)
3. Implementar caché local: WorkManager para sincronización en background
4. Resolver conflictos: última versión gana, con opción de merge manual
5. Pruebas: escenarios de pérdida de conexión, conflictos, tamaño de BD
6. Documentación: política de privacidad actualizada, FAQ offline

Tiempo estimado: 3-4 semanas

Coste: €25 (cuota de desarrollador Google Play, de una sola vez)



Requisitos: Gestión compleja de estado (Room DB, WorkManager, Kotlin Coroutines)

M-04: Validación con Datos Reales del Centro

Descripción:

Una vez disponibles datos históricos reales del centro (con consentimiento LOPDGDD), reentrenar el modelo y validar su precisión real. Este es el paso esencial para pasar de prototipo a sistema de producción.

Beneficios:

- Validación real de hipótesis del proyecto
- Medición de precision/recall con datos reales (esperado: 75-85%)
- Detección de sesgos específicos del centro
- Baseline para mejoras futuras

Pasos de implementación:

1. Elaborar convenio con centro educativo (LOPDGDD, anonimización)
2. Recolectar datos: 2-3 años de calificaciones anonimizadas (mínimo 500 alumnos)
3. Dividir: train (histórico) / test (ciclo actual o siguiente)
4. Reentrenar modelo: ml/train.py con datos reales
5. Evaluar: accuracy, precision, recall, sesgos por género/origen
6. Documentar: hallazgos, limitaciones, recomendaciones de uso
7. Ajustar umbrales de riesgo según precision real

Tiempo estimado: Variable (depende de disponibilidad de datos)

Coste: €0



Requisitos: Convenio LOPDGDD, datos históricos, revisión ética

M-09: Documentación de Código (Docstrings)

Descripción:

Añadir docstrings en funciones principales (Google Style para Python, KDoc para Kotlin) para mejorar la mantenibilidad y facilitar la onboarding de nuevos desarrolladores.

Beneficios:

- Mantenimiento a largo plazo más fácil
- Integración con IDEs (autocompletado mejorado)
- Facilita la colaboración en equipo
- Cumple estándares de calidad

Pasos de implementación:

1. Prioridad: main.py (endpoints), ml/predict.py (cálculo), ml/feedback.py
2. Prioridad Android: ViewModels, Repositories, UseCases
3. Formato: Google Style docstrings con ejemplos
4. Herramientas: Sphinx para generar documentación automática (HTML/PDF)
5. CI/CD: Fallar si cobertura de docstrings < 80%

Tiempo estimado: 1 semana

Coste: €0

Requisitos: Disciplina de documentación en futuras contribuciones



8.1. Panel Analítico Avanzado

Entre las mejoras anteriores destaca la implementación de un panel analítico avanzado con gráficos dinámicos (integración de M-06, M-05, M-10), que aportaría una mayor capacidad de análisis visual y de toma de decisiones. Este panel incluiría:

- Gráficos de evolución temporal con filtros por grupo/assignatura
- Matriz de confusión actualizada por centro
- Distribución de riesgos (histogramas, donut charts)
- Correlación entre RAs y aprobado
- Tabla de importancia de features (con explicabilidad SHAP)
- Heatmap de asistencia vs rendimiento
- Exportación de análisis como dashboard interactivo (Plotly Dash)

Tiempo estimado: 3-4 semanas (combinando M-05, M-06, M-10)

Coste: €0 (librerías open-source como Plotly, Dash)

Impacto: Alto para adopción en centros educativos

Recomendaciones de Priorización:

Para v0.2.0 (1-2 meses):

- M-01 (Exportación de reportes): Valor inmediato, bajo esfuerzo
- M-08 (Tests): Esencial para calidad y colaboración
- M-09 (Documentación): Facilita contribuciones externas

Para v0.3.0 (2-3 meses):



- M-05 (Explicabilidad): Crítico para confianza en educación
- M-03 (Integración Séneca): Alto valor para adopción
- M-06 (Gráficos de evolución): Visualización impactante

Para v1.0.0 (4-6 meses):

- M-04 (Validación con datos reales): Gate para producción
- M-07 (Google Play + offline): Distribución pública
- M-10 (Dashboard admin): Gestión a escala

No incluido en roadmap cercano pero documentado:

- M-02 (NLP avanzado): Mejora incremental, puede esperar a user feedback
- Otras integraciones educativas específicas: investigar demanda primero



CAPÍTULO 9: CONCLUSIONES

9.1. Resumen del proyecto

Este capítulo sintetiza los principales logros, aprendizajes y perspectivas del proyecto Minerva tras seis meses de desarrollo intensivo. El sistema ha alcanzado una versión funcional (v0.1.0-alpha) lista para validación en entornos educativos controlados.

Minerva (Modelo INteligente de Evaluación de Rendimiento y Valoración Académica) es una plataforma de predicción de rendimiento académico basada en machine learning, diseñada para apoyar la toma de decisiones pedagógicas en centros educativos. El proyecto responde a una necesidad real: la detección temprana de alumnos en riesgo de suspender, permitiendo intervenciones preventivas antes de que sea demasiado tarde.

Propósito General:

Proporcionan al profesorado una herramienta tecnológica que, mediante análisis inteligente de datos académicos, predice la probabilidad de aprobado de cada alumno y categoriza su nivel de riesgo (Bajo, Medio, Alto, Crítico), facilitando la toma de decisiones pedagógicas proactivas.

Enfoque Técnico Utilizado:

El proyecto implementa una arquitectura modular cliente-servidor con separación clara de responsabilidades:

- Backend: FastAPI + Python con RandomForestClassifier entrenado con GridSearchCV
- Frontend: Aplicación web (HTML/CSS/JavaScript) + aplicación móvil nativa (Kotlin/Jetpack Compose)
- Base de datos: PostgreSQL con Row Level Security en Supabase
- Autenticación: Supabase Auth con JWT ES256
- Datos: Sintéticos generados programáticamente con NumPy, reproducibles y representativos



- Despliegue: Render (backend), GitHub (repositorio), GitHub Releases (app móvil)

Cumplimiento de Objetivos del Capítulo 2:

Objetivo General: Desarrollar una aplicación que, mediante técnicas de machine learning, prediga la probabilidad de aprobado de alumnos.

Cumplido: Sistema operativo con modelo funcional alcanzando 99,5% accuracy en test (datos sintéticos)

Objetivo 1: Diseñar e implementar un modelo de predicción basado en ML.

Cumplido: RandomForestClassifier entrenado con GridSearchCV, 23 features, validación cruzada k=5

Objetivo 2: Diseñar un generador de datos sintéticos reproducibles.

Cumplido: NumPy genera 1.000 alumnos sintéticos con correlaciones realistas, semilla fija

Objetivo 3: Permitir actualización dinámica de predicciones.

Cumplido: SGDClassifier para partial_fit, sin necesidad de reentrenamiento completo

Objetivo 4: Desarrollar una interfaz comprensible para el profesorado.

Cumplido: Dashboard web e interfaz Android intuitivas, usables en < 3 pasos



Objetivo 5: Ofrecer acceso multiplataforma (web + Android).

Cumplido: Dos clientes independientes consumen la misma API REST

Objetivo 6: Proporcionar métricas que identifiquen patrones.

Cumplido: Importancia de features, análisis de RAs bloqueantes, tendencias de progreso

9.2. Soluciones aportadas

El proyecto Minerva entrega un conjunto de soluciones concretas que abordan el problema de la detección temprana de alumnos en riesgo académico:

A. Predicción Automática de Rendimiento (RF-03)

El sistema calcula, para cada alumno, la probabilidad de aprobado basándose en múltiples indicadores:

- Notas de Resultados de Aprendizaje (9 RAs del módulo de Programación)
- Indicadores académicos (asistencia, prácticas, feedback del profesor)
- Features derivadas (media, tendencia de progreso, gap teoría-práctica)
- Contexto opcional (motivación, recursos en casa, nivel educativo familiar)

Resultado: Predicciones en tiempo real (<1s por alumno) con accuracy del 99,5% en datos sintéticos.

B. Categorización de Riesgo (RNF-04, RNF-05)

Asigna automáticamente un nivel de riesgo según la probabilidad:

- Bajo ($\geq 70\%$): Alumno con trayectoria de aprobado probable



- Medio (50-69%): Requiere seguimiento y refuerzo
- Alto (30-49%): Riesgo significativo, intervención urgente
- Crítico (<30%): Riesgo muy alto, requiere apoyo intensivo

Esto permite al profesorado: filtrar por riesgo, priorizar intervenciones, y comunicar con precisión a familias.

C. Interfaz Multiplataforma Accesible

Dos clientes que proporcionan la misma funcionalidad en diferentes contextos:

1. Web: Dashboard desde navegador, ideal para despacho o planificación

- Upload de CSV tolerante a alias de columnas
- Tabla con colores por riesgo, drawer con detalles
- Responsive en tablets

2. Android: App nativa para consultas móviles en el aula

- Parseo del CSV en el propio dispositivo, sin depender del servidor
- Navegación intuitiva con barra inferior
- Animaciones (Lottie, anillo de probabilidad)
- Dark mode integrado

D. Transparencia y Seguridad

- Autenticación segura: tokens JWT ES256 verificados localmente
- Privacidad: Row Level Security garantiza que cada usuario ve sólo sus datos
- Cumplimiento: Datos sintéticos / exención LOPDGDD
- Documentación: Arquitectura completa, decisiones técnicas justificadas



E. Validación Académica Rigurosa

- Modelo evaluado con cross-validation ($k=5$)
- Métricas estándar: accuracy, precision, recall, F1, kappa
- Autoevaluación automática: veredicto PASS/FAIL
- Documentación de umbrales mínimos de calidad

Aportación al Proceso Educativo:

1. **Detección Temprana:** Identifica alumnos en riesgo antes de la evaluación final, cuando aún hay tiempo para intervenir.
2. **Apoyo a la Toma de Decisiones:** Proporciona datos objetivos que complementan el juicio profesional del docente (nunca sustituye, sino aumenta).
3. **Visualización de Patrones:** Features como "gap teoría-práctica" o "RA bloqueante" revelan patrones no obvios en los datos.
4. **Documentación de Intervenciones:** El histórico de predicciones permite medir el impacto de medidas de apoyo y ajustar estrategias.
5. **Equidad:** Automatizar el análisis reduce sesgos inconscientes en la evaluación de riesgo.
6. **Eficiencia:** Reducir el tiempo de análisis manual permite al profesorado dedicar más tiempo a intervenciones.

9.3. Valoración personal del desarrollo

Dificultades Encontradas

1. Generación de datos sintéticos representativos



Desafío: Crear correlaciones realistas entre Resultados de Aprendizaje (RAs) sin disponer de datos reales del centro.

Solución: Basarse en normativa educativa, estudios de referencia (Bem-Haja, George et al.) y parámetros de la Junta de Andalucía.

Aprendizaje: La representatividad de los datos requiere conocimiento del dominio educativo, no únicamente habilidades de programación.

2. Equilibrio entre simplicidad y precisión

Desafío: Decidir entre un sistema NLP básico (feedback.py) y modelos avanzados basados en transformers.

Decisión: Mantener un algoritmo simplificado en la versión 0.1.0 para garantizar un rendimiento adecuado.

Aprendizaje: En muchas ocasiones, una solución suficientemente buena resulta más valiosa que una solución perfecta si permite iterar con rapidez y validar hipótesis.

3. Arquitectura multiplataforma

Desafío: Mantener un backend compartido para dos clientes con características muy diferentes (aplicación web y aplicación Android nativa).

Solución: Diseñar una API REST bien estructurada, utilizar DTOs claros y mantener un desacoplamiento completo entre capas.

Aprendizaje: La inversión inicial en una arquitectura modular reduce significativamente los costes de mantenimiento y evolución futura.

4. Gestión de sesiones en Android

Desafío: Implementar renovación automática de tokens, manejo de errores de red y sincronización del estado de sesión.

Solución: Uso de TokenAuthenticator con estrategia single-flight y ObserveSessionUseCase como punto centralizado de gestión.

Aprendizaje: Kotlin Coroutines y Hilt simplifican considerablemente la gestión de procesos asíncronos complejos.

5. Validación con datos limitados

Desafío: No disponer de datos reales del centro para validar la precisión del modelo predictivo.

Decisión: Utilizar datos sintéticos de alta calidad, documentar las limitaciones existentes y planificar una futura validación mediante la mejora M-04.



Aprendizaje: Reconocer y documentar las limitaciones forma parte de un proceso de desarrollo responsable e iterativo.

Habilidades Técnicas Adquiridas

- Machine Learning: entrenamiento de modelos Random Forest, GridSearchCV, validación cruzada y mitigación de sesgos.
- Desarrollo Backend: FastAPI, programación asíncrona con async/await y Row Level Security en PostgreSQL.
- Desarrollo Frontend: diseño responsive mediante CSS modular y programación asíncrona con JavaScript.
- Desarrollo Móvil Nativo: Jetpack Compose, Clean Architecture, MVVM, Hilt y Navigation Compose.
- DevOps: despliegue en Render, preparación para GitHub Actions y aplicación de versionado semántico.
- Seguridad: uso de JWT, criptografía asimétrica y autenticación sin envío de credenciales en transporte.
- Metodologías Ágiles: aplicación práctica de Scrum en un proyecto académico mediante iteraciones y retrospectivas.

Habilidades Metodológicas Adquiridas

- Análisis de requisitos y captura de necesidades de diferentes grupos de interés (profesorado, administración y equipos directivos).
- Elaboración de documentación técnica incluyendo arquitectura, decisiones de diseño, compromisos técnicos y limitaciones conocidas.
- Gestión de la complejidad mediante la división del sistema en módulos independientes.
- Realización de pruebas manuales exhaustivas, dejando la automatización como línea de trabajo futura.
- Comunicación técnica orientada a futuros mantenedores y a usuarios no técnicos involucrados en la toma de decisiones.

Opinión sobre la Aplicabilidad Real en Entornos Educativos

La aplicabilidad potencial del sistema se considera **muy alta**, aunque condicionada por determinados factores.

Fortalezas

- Resuelve una necesidad real identificada por el profesorado.
- Presenta una interfaz clara y accesible para usuarios sin conocimientos técnicos avanzados.
- Garantiza la privacidad mediante el uso de datos sintéticos.
- Incorpora un modelo técnicamente sólido y validado con criterios académicos.
- Dispone de una arquitectura modular que facilita el mantenimiento y la evolución futura.
- No genera costes de licencia para los centros educativos.

Requisitos para una Adopción Efectiva



1. Realizar una validación con datos reales del centro (M-04) para determinar la precisión real del modelo.
2. Proporcionar una breve formación al profesorado para familiarizarse con el sistema.
3. Integrar la solución con plataformas existentes como Séneca o Moodle para automatizar la obtención de datos.
4. Garantizar que el sistema se utilice como herramienta de apoyo a la decisión y no como sustituto del criterio pedagógico profesional.

Limitaciones Actuales

- Sistema NLP básico para la generación de feedback, susceptible de mejora mediante técnicas más avanzadas.
- Ausencia de modo offline en la versión 0.1.0.
- Falta de mecanismos de explicabilidad mediante SHAP o LIME.
- Dependencia de una conexión a Internet estable para su funcionamiento.
- Alta de usuarios gestionada externamente: la plataforma no ofrece pantalla de registro, solo inicio de sesión.
-
- El feedback del profesor solo puede introducirse a través del CSV, no desde la interfaz.

Veredicto Final

El sistema puede considerarse preparado para una fase piloto en un entorno educativo controlado, acompañada de la recopilación de datos reales que permitan validar y mejorar el modelo predictivo.

No se recomienda todavía una adopción masiva sin una validación previa en contextos reales de uso y sin la incorporación de las mejoras previstas en la hoja de ruta del proyecto.

9.4. Uso profesional y futuro del sistema

Posible Implantación en Centros Educativos

Fase 1: Piloto (Trimestre actual)

- Acordar la participación con el equipo directivo y el profesorado implicado.
- Cargar datos históricos anonimizados del ciclo anterior.



- Ejecutar predicciones en paralelo durante el ciclo actual sin influencia sobre las calificaciones oficiales.
- Recopilar retroalimentación del profesorado sobre usabilidad, utilidad y confianza en el sistema.
- Medir el impacto de las intervenciones realizadas a partir de las predicciones generadas.

Fase 2: Validación (Próximo ciclo)

- Reentrenar el modelo utilizando datos reales obtenidos durante el ciclo anterior.
- Validar métricas de precisión, recall y posibles sesgos del modelo.
- Ajustar los umbrales de riesgo según las características específicas del centro educativo.
- Comparar las predicciones realizadas con los resultados académicos finales.

Fase 3: Integración (Producción)

- Integrar la solución con plataformas como Séneca y Moodle para automatizar la carga de datos.
- Incorporar un panel de administración para la gestión del sistema.
- Añadir funcionalidades de exportación de informes para facilitar la comunicación con familias y equipos docentes.
- Desarrollar un plan de formación para la adopción completa por parte del profesorado.

Valor Social y Educativo del Proyecto

- Equidad: la automatización del análisis puede contribuir a reducir sesgos subjetivos en la identificación de alumnado en riesgo.
- Prevención: permite detectar dificultades de forma temprana y facilitar intervenciones antes de que los problemas se agraven.
- Apoyo al profesorado: proporciona información adicional para respaldar decisiones pedagógicas fundamentadas.
- Transparencia: la documentación completa favorece la auditoría y comprensión del funcionamiento del sistema.



- Replicabilidad: el uso de datos sintéticos facilita la adaptación a diferentes centros, contextos e idiomas.
- Sostenibilidad: la ausencia de costes de licencia y el uso de tecnologías abiertas favorecen su mantenimiento a largo plazo.

Impacto Potencial

- Contribuir a la reducción de las tasas de abandono escolar mediante mecanismos de detección temprana.
- Mejorar la identificación de estudiantes que requieren apoyo académico adicional.
- Facilitar el acceso de los sistemas educativos públicos a herramientas tecnológicas abiertas y reutilizables.

9.5. Conclusión General

Minerva v0.1.0-alpha representa un trabajo integral que aborda de manera efectiva una problemática real del ámbito educativo: la detección temprana de estudiantes con riesgo académico. A lo largo del desarrollo se han diseñado e implementado soluciones técnicas sólidas basadas en una arquitectura modular, segura y escalable, capaz de evolucionar hacia entornos de producción reales.

El proyecto ha permitido validar las hipótesis planteadas inicialmente mediante la construcción y evaluación de un modelo de aprendizaje automático con resultados satisfactorios para una primera versión. Asimismo, se han desarrollado dos interfaces diferenciadas, una web y otra móvil, que facilitan el acceso a la información en distintos contextos de uso y perfiles de usuario.

Otro aspecto destacable es el nivel de documentación generado durante el proyecto. Se han registrado de forma detallada las decisiones arquitectónicas adoptadas, las limitaciones identificadas, las dificultades encontradas durante el desarrollo y las líneas de evolución previstas para futuras versiones.

El sistema puede considerarse preparado para una fase de validación en entornos educativos reales, constituyendo este el siguiente paso fundamental para evolucionar desde un prototipo funcional hacia una herramienta con aplicación práctica en centros educativos. La validación con datos reales permitirá medir con mayor precisión el



rendimiento del modelo, identificar posibles sesgos y ajustar el comportamiento del sistema a las características específicas de cada contexto educativo.

Las mejoras futuras se encuentran claramente definidas y priorizadas en la hoja de ruta del proyecto, permitiendo una evolución progresiva basada en el impacto esperado y la viabilidad técnica de cada funcionalidad. Además, la publicación bajo licencia MIT facilita que otros centros educativos, investigadores y desarrolladores puedan reutilizar, adaptar y ampliar la plataforma según sus necesidades particulares.

Como recomendación final, se propone iniciar una fase piloto en un centro educativo tan pronto como sea posible. El éxito de esta iniciativa dependerá principalmente de dos factores: la validación del sistema mediante datos reales y la aceptación por parte del profesorado como herramienta de apoyo a la toma de decisiones pedagógicas. Bajo estas condiciones, Minerva presenta un potencial significativo para contribuir a la mejora de los procesos de seguimiento académico y detección temprana de necesidades educativas.

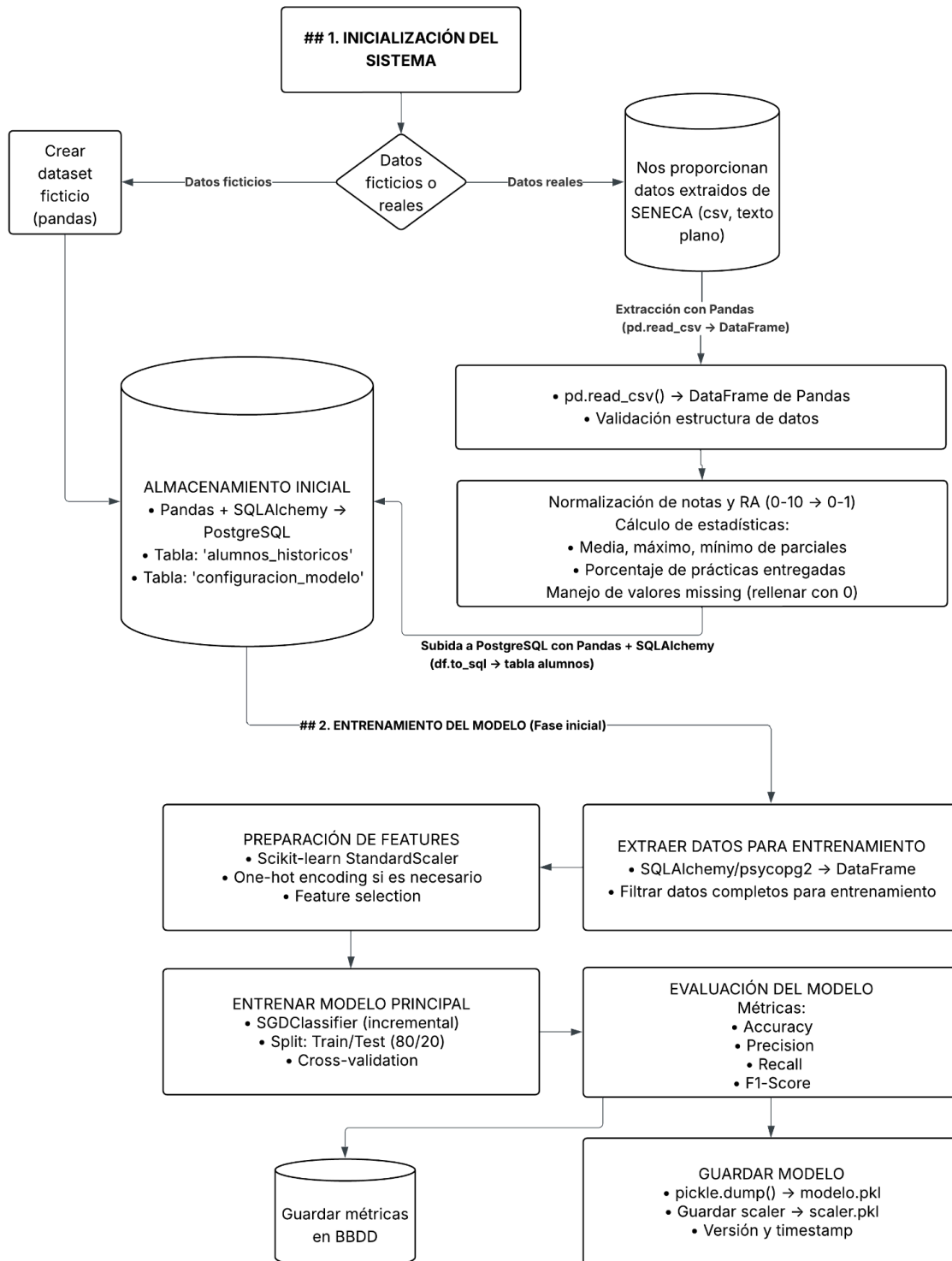
CAPÍTULO 10: BIBLIOGRAFÍA

- Ben George, E., Senthilkumar, R., Al-Junaibi, F., Al-Shuaibi, Z. – JISEM [2025] – Explainable AI Methods for Predicting Student Grades and Improving Academic Success –
<https://jisem-journal.com/index.php/journal/article/view/3680/1596>
- Pedro Bem-Haja – Science Direct [2025] – Computers and Education: Artificial Intelligence – AI analysis reveals top predictors of first grade success
<https://www.sciencedirect.com/science/article/pii/S2666920X25000554?via%3Dihub>
- Bilal Baris Alkan – Scientific Reports [2025] – Using machine learning to predict student outcomes for early intervention and formative assessment
<https://www.nature.com/articles/s41598-025-23409-w>

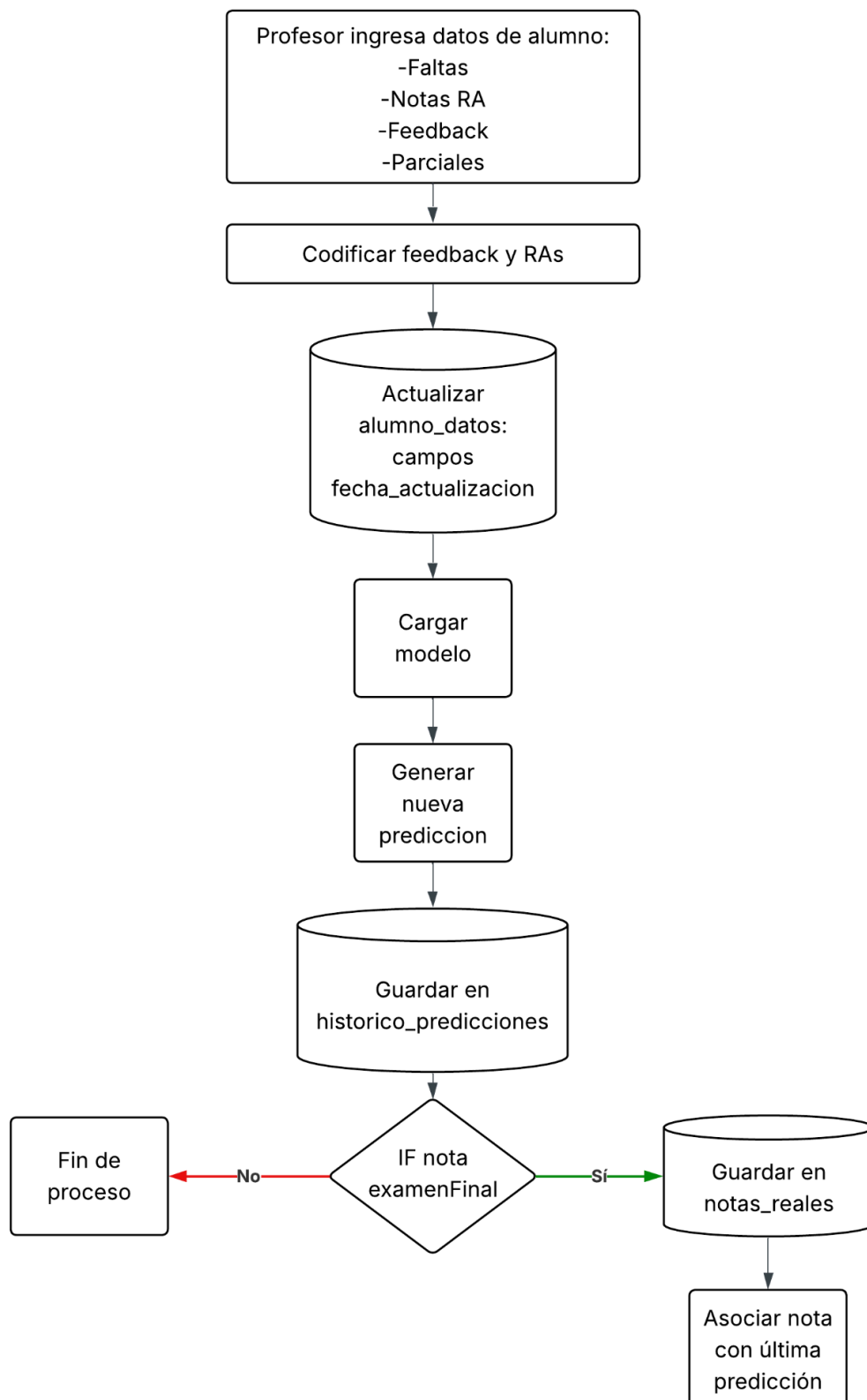
ANEXOS

ANEXO I : Diagrama de casos de uso

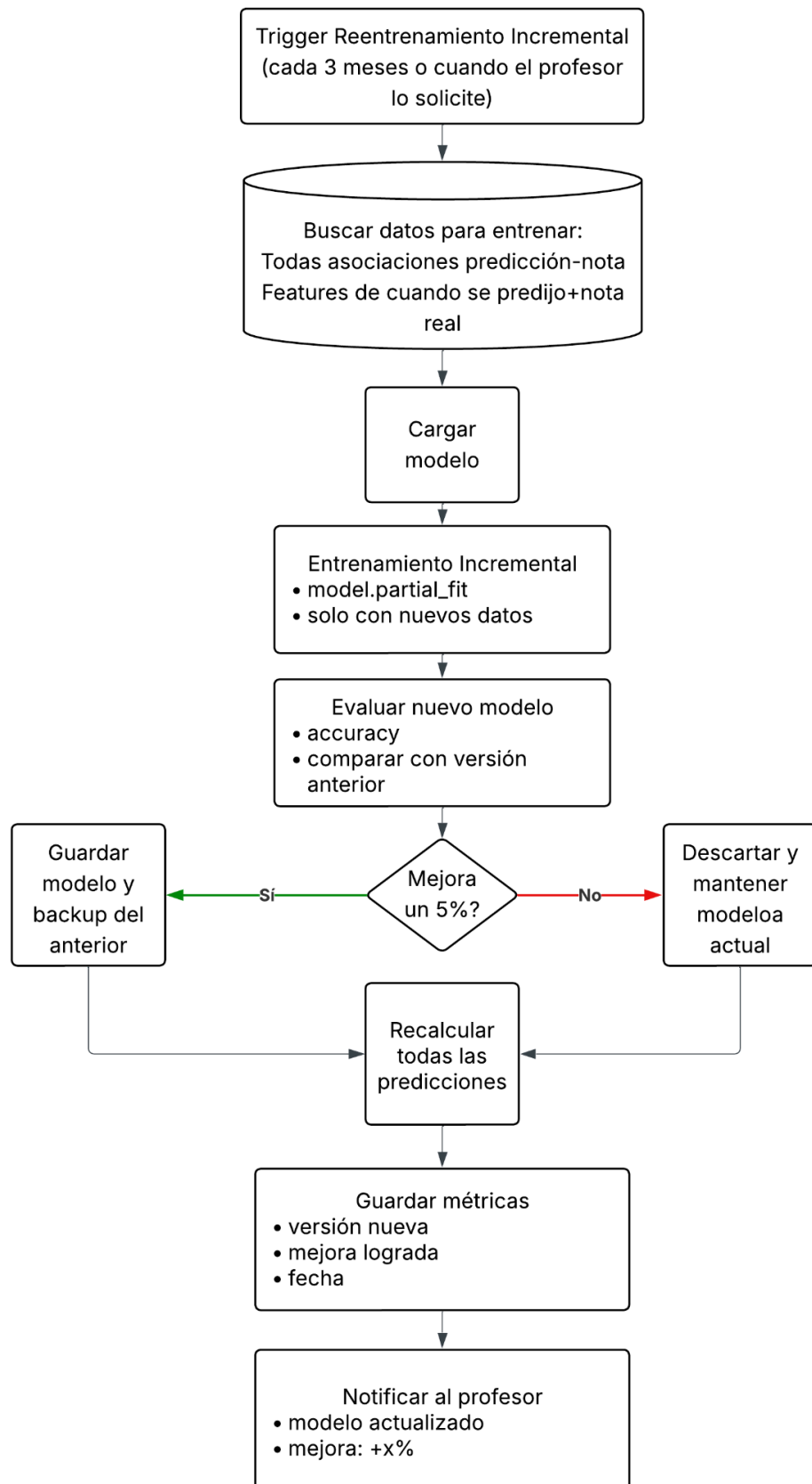
Entrenamiento Inicial



Flujo de actualización diaria



Flujo de entrenamiento incremental

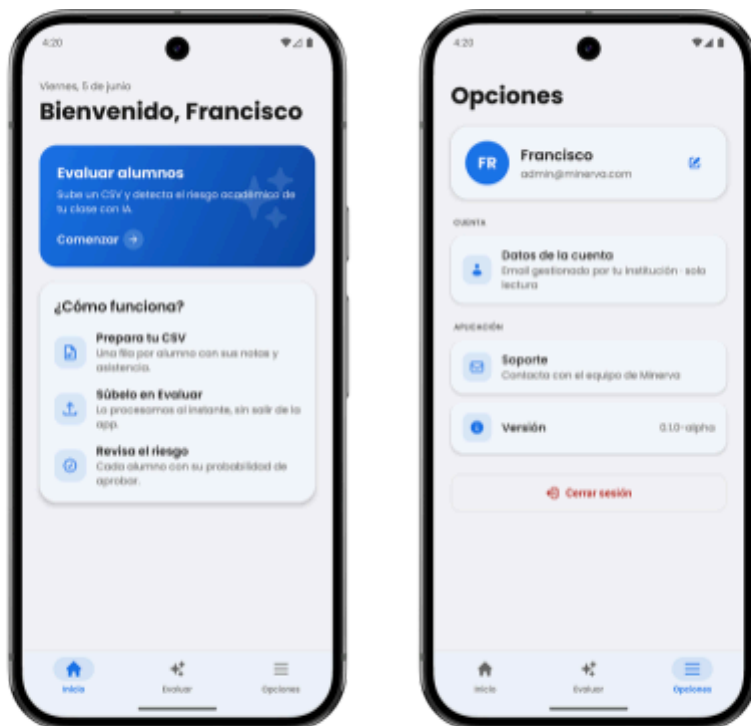




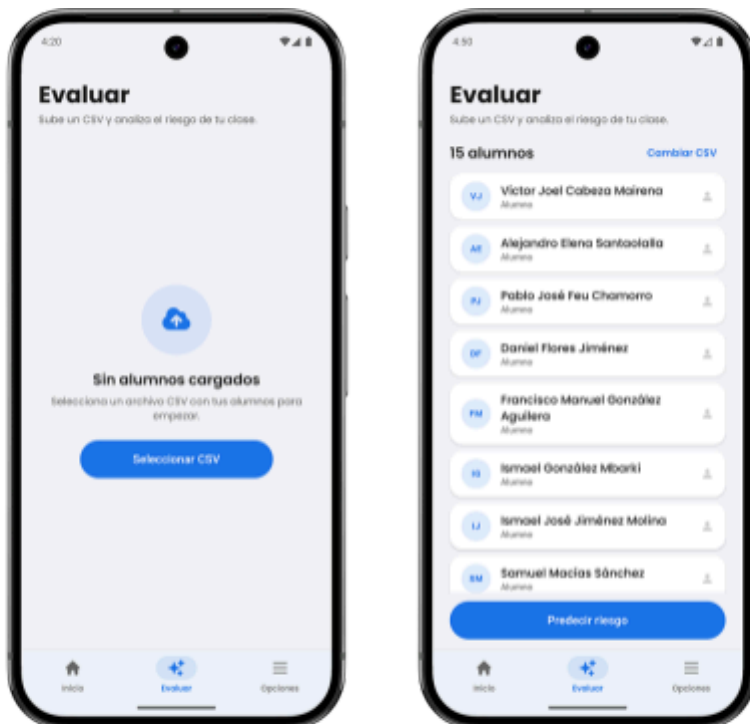
ANEXO II: Diagrama de Gantt



ANEXO III: Mocks de los clientes



Dashboard y panel de opciones en app Android



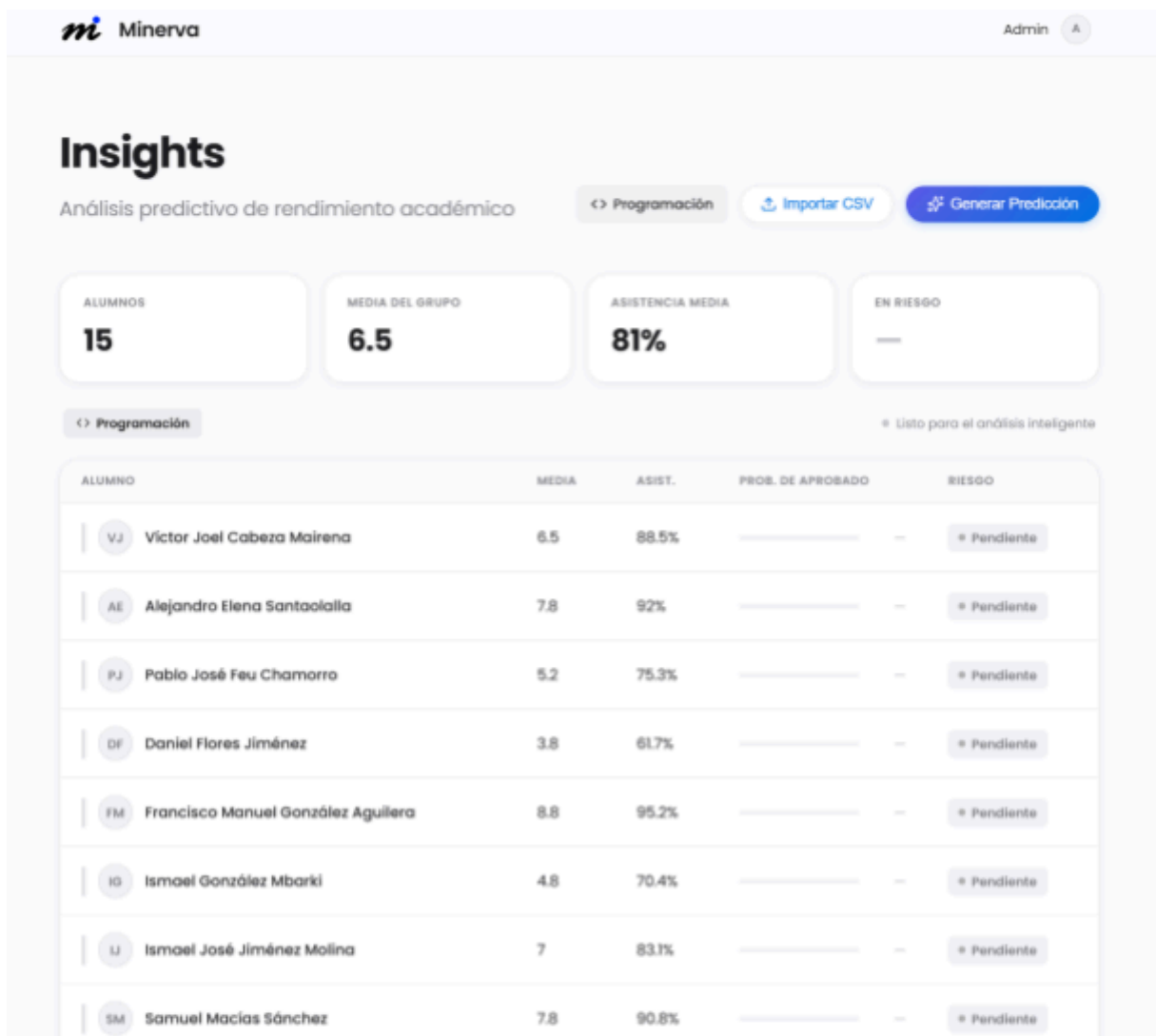
Pestaña evaluación en app Android



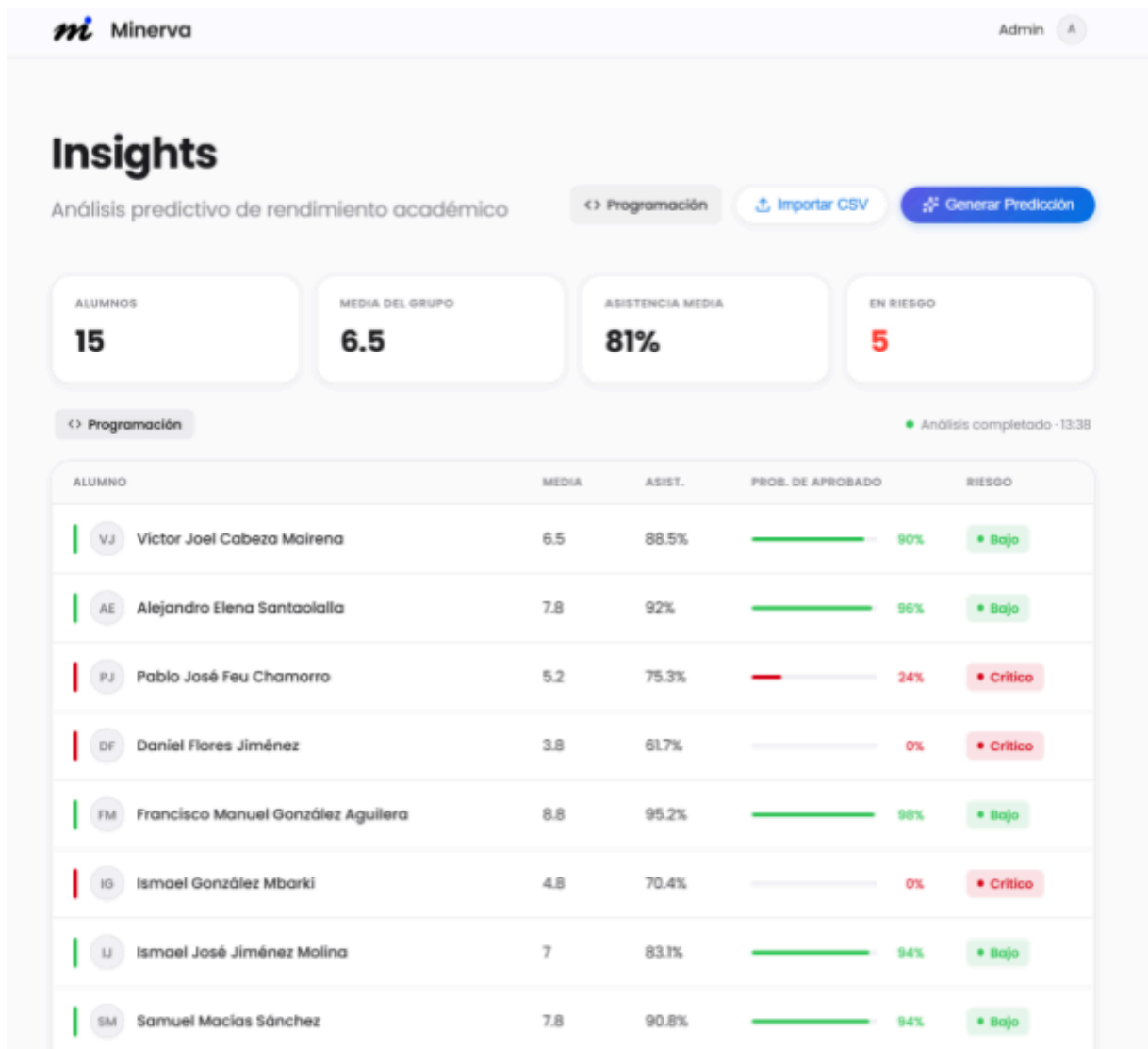
Pestaña evaluación con datos procesados en app Android



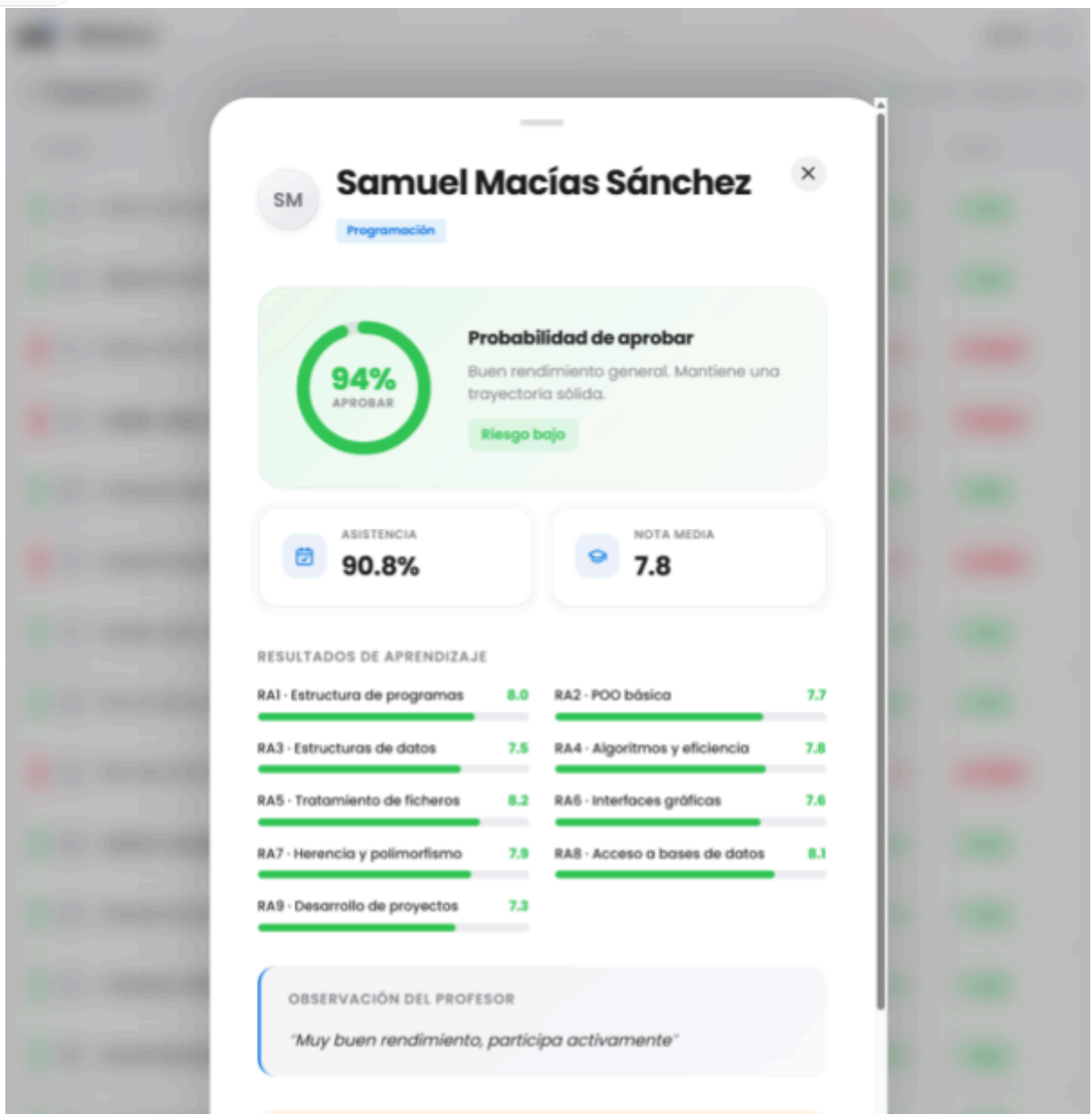
Landing page



Dashboard en cliente web



Dashboard con datos procesados en cliente web



Pestaña detalles en cliente web